

מבוא לבינה מלאכותית - חוברת מערכי תרגול תשפ"ו.

רגב יחזקאל אימרה.

12 בינואר 2026

קישור לגיט עם כל הקוד ומערכי התרגול - <https://github.com/Regev32/Intro-to-artificial-intelligence>

תוכן עניינים

2	I מודלי רגרסיה.	1
2	רגרסיה לינארית.	1
3	1.1 אלגוריתם GD.	
4	1.2 אלגוריתם SGD.	
5	2 מטריקות הערכת המודל.	
6	3 רגרסיה פולינומאלית.	
7	4 BIAS VARIANCE TRADEOFF.	
8	5 BIC.	
8	6 רגולריזציה.	
8	6.1 RIDGE.	
9	6.2 LASSO.	
10	6.3 ELASTIC NET.	
10	6.4 EARLY STOPPING.	
11	6.5 סטנדרטיזציה.	
11	II מודלי סיווג.	7
12	רגרסיה לוגיסטית.	7
13	7.1 פונקציית loss.	
14	7.2 איך מאמנים?	
15	7.3 גבול ההחלטה.	
15	7.4 רגולריזציה.	
16	8 SVM (מכונת וקטורים תומכים).	
18	8.1 פונקציית loss.	
19	8.2 איך מאמנים?	
20	9 Kernel-SVM - כשהדאטה לא פריד לינארית.	
22	10 סיווג לא בינארי Multi-class Classification.	
22	10.1 סיווג One vs. All.	
24	10.2 סיווג One vs. One.	
24	10.3 סיווג Softmax regression.	
25	10.3.1 פונקציית loss.	
25	10.4 איך מאמנים?	
28	11 הערכת המודל המסווג.	
28	11.1 מטריצת בלבול - Confusion Martix.	
29	11.2 ROC, AUC ושאר חבריהם.	
32	11.3 הערכת מודל מסווג לא בינארי.	
33	12 אלגוריתם KNN.	
34	13 קביעת היפרפרמטרים ו-Cross-validation.	
35	III רשתות נוירונים.	14
35	14 הגדרות, אינטואיציה.	
38	15 Back-propagation.	
40	16 אימון רשתות (עמוקות).	
40	16.1 Vanishing\Exploding gradients.	

40		Weight Initialization	16.1.1
41		Batch Normalization	16.1.2
41		Gradient Clipping	16.1.3
41		Dropout - רגולריזציה ברשתות נוירונים	16.2
41		Optimizers	17
41		Momentum	17.1
42		Nestrov Momentum	17.2
42		AdaGrad	17.3
43		RMSProp	17.4
43		Adam	17.5

הקדמה:

יש הרבה סוגי למידה בעולם, הנה חלקם:

- למידה מפקחת: לכל דגימה יש את התווית שלה (label).
 - (א) סיווג: חיזוי קטגוריות.
(ב) רגרסיה: חיזוי ערך רציף.
 - למידה לא מפקחת: יש דגימות, אין תוויות.
(א) אשכול: חלוקת דגימות לקבוצות.
(ב) הורדת מימדים: הפתחת כמות הפיצ'רים של המודל.
(ג) זיהוי אנומליות: גילוי דגימות חריגות מבין סך כל הדגימות.
 - למידה באמצעות חיזוקים: המודל לומד דרך ניסוי וטעייה. המודל מבצע פעולות ומקבל חיזוקים (חיוביים או שליליים) ומשפר את המדיניות שלו.
 - למידה מונחית עצמית: המערכת מייצרת לעצמה תוויות מתוך הנתונים כדי ללמוד ייצוגים יותר טובים.
 - למידה גנרטיבית: המערכת מייצרת נתונים חדשים הדומים לנתונים המקוריים עליהם אומנה.
- בקורס הזה נתמקד בעיקר בלמידה מפקחת. למעוניינים, המחלקה למתמטיקה מציעה קורס בלמידה לא מפקחת, והמחלקה למדעי המחשב מציעה קורסים בשאר סוגי הלמידה.

חלק I

מודלי רגרסיה.

1 רגרסיה לינארית.

לאילו שלא זוכרים, בלמידה מפקחת יש שתי נישות מרכזיות: קלסיפיקציה ורגרסיה.

הגדרה 1.1. מודל **רגרסיה** הוא מודל סטטיסטי הנועד להעריך קשר בין משתנים.

דוגמה 2.1. דוגמאות למודל רגרסיה:

- חיזוי מחיר של בית לפי גודל ומיקום.
- חיזוי ציון של סטודנט לפי מספר שעות למידה.
- חיזוי הטמפרטורה של מחר על סמך נתוני מזגל האוויר של הימים האחרונים.

איך עובד מודל רגרסיה?

(1) הנחות המודל: נניח יש לנו אוסף של N דגימות והתייגים שלהן, $\{(x_i, y_i)\}_{i=1}^N$ כאשר $x_i \in \mathbb{R}^d$ וכן $y_i \in \mathbb{R}$. רגרסיה לינארית, כשמה כן היא, מניחה כי הקשר בין x ו- y הוא לינארי, כלומר שקיימים $d+1$ סקלרים $b, \{w_i\}_{i=1}^d$ כך שמתקיים $y_i = \sum_{j=1}^d x_{ij}w_j + b = w^T x_i + b$. נשאלת השאלה, איך אנחנו מגלים מהם הסקלרים w_i, b הנ"ל?

(2) פונקציית הפסד: לכל דגימה (x_i, y_i) המודל חוזה ערך כלשהו $\hat{y}_i$. נגדיר את השגיאה של המודל להיות $error_i = y_i - \hat{y}_i$. אנחנו מעוניינים במספר אחד שיהיה תלוי בכל השגיאות שלנו ושיגיד לנו כמה רע המודל שלנו. יש לכך כמה בחירות טבעיות:

- סכום השגיאות: $\sum (y_i - \hat{y}_i)$. הבעיה היא שפה אם יש לי שגיאה חיובית ושגיאה שלילית הן יתבטלו, מה שיכול להוביל לדיווחים שגויים.
- סכום ערכים מוחלטים: $\sum |y_i - \hat{y}_i|$. יותר טוב, אבל פה הבעיה היא שזה לא גזיר - הרי אם אנחנו רוצים למזער את השגיאה הכוללת שלנו נצטרך לגזור אותה.
- סכום השגיאות בריבוע: $\sum (y_i - \hat{y}_i)^2$. מעולה בשבילנו. זה גזיר ונותן יותר משקל ככל שהשגיאה גדלה.

הגדרה 3.1. פונקציית הפסד (loss function) היא פונקציה הממפה את התחזית של המודל ואת הערך האמיתי למספר ממשי, המייצג את מידת הטעות או העלות של התחזית.

באופן טבעי, ככל שפונקציית ההפסד מניבה ערך נמוך יותר עבור תחזיות המודל, כך ביצועי המודל טובים יותר. מטרתנו היא למזער את ערך פונקציית ההפסד.

הגדרה 4.1. נגדיר את ה-loss function **טעות הריבועית הממוצעת** (MSE) להיות

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

כאשר θ הם הפרמטרים של המודל שלנו שאנחנו מעוניינים ללמוד (ללמוד = למזער את ה-loss שלנו), ו- $\hat{y}_i = f(x_i; \theta)$ זה הערך שהמודל פולט לדגימה x_i עם פרמטרים θ . נרצה למצוא $\hat{\theta} = \arg \min_{\theta} \mathcal{L}(\theta)$.

זוהי פונקציית הפסד סטנדרטית בבעיות רגרסיה בכללית לאו דווקא ברגרסיה לינארית.

(3) תהליך הלמידה: נציין לפרוטוקול כי קיים פתרון סגור לבעיה הזו: נכתוב את הדאטה שלנו בכתוב מטריוני:

$$X = \begin{pmatrix} 1 & - & x_1^T & - \\ 1 & - & x_2^T & - \\ \vdots & & \vdots & \\ 1 & - & x_N^T & - \end{pmatrix} \bullet X \in \mathbb{R}^{N \times (d+1)} \text{ המוגדר להיות}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \bullet y \in \mathbb{R}^N \text{ כל הערכים שלנו בצורת וקטור}$$

$$\beta = \begin{pmatrix} b \\ w_1 \\ \vdots \\ w_d \end{pmatrix} \bullet \beta \in \mathbb{R}^{d+1} \text{ כל הפרמטרים שאנחנו רוצים ללמוד}$$

כלומר המודל שלנו הופך להיות

$$\hat{y} = X\beta$$

ופונקציית ההפסד שלנו היא

$$\mathcal{L}(\beta) = \frac{1}{N} \|y - X\beta\|_2^2$$

כדי למזער, נגזור ביחס ל- β ונשווה לאפס:

$$\nabla_{\beta} \mathcal{L} = -\frac{2}{N} X^T (y - X\beta)$$

כלומר נקבל

$$X^T y = X^T X \beta$$

ולפי גאוס הפתרון יהיה

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

הבעיה: לרוב, המטריצה $X^T X$ לא הפיכה, לכן נצטרך להיות יותר חכמים בחיפוש שלנו אחרי $\hat{\beta}$. הפתרון: ניעזר באלגוריתם מורד הגרדיאנט

1.1 אלגוריתם GD.

אלגוריתם 5.1. אלגוריתם מורד הגרדיאנט - GD:

- בחרו נקודת התחלה θ_0 .
- חשבו את $\nabla_{\theta} \mathcal{L}(\theta_0)$.
- בחרו גודל צעד $\eta > 0$.
- חשבו $\theta_{k+1} = \theta_k - \eta \nabla_{\theta} \mathcal{L}(\theta_k)$.

$$\theta^+ = \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

מוטיבציה: הגרדיאנט של פונקציה הוא כיוון העליה הכי מהיר שלה, לכן לחסר מהמיקום הנוכחי כפולה שלילית של הגרדיאנט תוריד אותנו אל עבר מינימום מקומי. לפיכך הוספה של כפולה שלילית של הגרדיאנט של פונקציית ההפסד ל- θ אמורה להוביל לאורך זמן לירידה בערך MSE. חשוב! צעד חשוב באלגוריתם מורד הגרדיאנט הוא בחירה נכונה של גודל הצעד: גודל צעד גדול מדי יגרום לכך שבחיים לא נתכנס, וגודל צעד קטן מדי יגרום לכך שנתכנס מקצב מאוד איטי (לצייר).

בדר"כ בוחרים $\eta = 0.05$, ואם צריך משנים את הערך במהלך האימון.

הגדרה 6.1. היפרמטר הוא פרמטר שקובעים על מנת להגדיר חלק כלשהו במודל.

במקרה שלנו קצב הלמידה הוא היפר פרמטר, כי הוא קובע כמה מהר מורד מגרדיאנט מתכנס ולכן הוא קובע האם יהיה למודל אימון איטי או מהיר.

הערה 7.1. בעיה מרכזית של מורד הגרדיאנט היא שאנחנו יכולים להיתקע במינימום מקומי (לצייר!) לא משנה כמה פעמים נריץ מורד הגרדיאנט. לכן ישבו אנשים חכמים והמציאו אלגוריתמים אחרים הידועים כתור *optimizers* (נרחיב עליהם בהמשך הקורב אולי), כגון

1. *SGD* שזה לעשות *GD* ולהוסיף רעש גאוסיאני לצעד שלנו.

2. *Momentum* שזה ככל שאנחנו מתקדמים בציר אחד יותר זמן, נתקדם בו יותר מהר.

3. *Nesterov* שזה כמו *Momentum* אבל קצת יותר חכם.

4. *AdaGrad* שמוריד את ה-*learning rate* ככל שמשיכים לאמן.

5. *RMS* שמוריד גם הוא את ה-*lr* אבל מונע מזה לקטון יותר מדי.

6. *Adam* שמשלב *RMS* ו-*Momentum*.

7. ולאחרונה גם *Moun*.

מסקנה: כלומר התחום עדיין פעיל ואין באמת שיטה הכי טובה למצוא את הפרמטרים הכי טובים אם אין פתרון סגור.

שאלה 8.1. מתי עוצרים את האימון?

פתרון: יש כמה אפשרויות:

1. כאשר $\|\nabla \mathcal{L}\| < \epsilon$.

2. נגמר התקציב- נניח מסיבה מסוימת יש לי רק שעה על המחשב להריץ את האלגוריתם- מה שיצא אני מרוצה.

3. אם חשוב להגיע לערך כלשהו של המודל (פרטים עוד רגע) ומצליחים, עוצרים.

4. כשעשיתי מספר צעדים שהגדרתי מראש.

1.2 אלגוריתם SGD.

למודנו על אלגוריתם מורד הגרדיאנט, שניתן לתארו באמצעות

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)$$

כאשר

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

היתרון הוא שכל צעד שאנחנו מתקדמים הוא מדויק אל עבר כיוון הירידה הכי טוב.

החסרונות הם:

1. אם יש לי N ממש ממש גדול, לעבור על כל הדגימות יקח נצח.

2. אני עלול להיתקע במינימום מקומי.

לכן אנחנו רוצים לשנות מעט את האלגוריתם כדי שיפתור את שני החסרונות האלו ושעדיין ישמור על היתרון. הדרך לעשות את זה היא באמצעות עדכון הגרדיאנט לפי קבוצה קטנה יותר של דגימות:

$$\bar{\mathcal{L}}(\theta) = \frac{1}{k} \sum_{j=1}^k (y_{i_j} - \hat{y}_{i_j})^2$$

כאשר $k \ll N$, ובכך לשנות את כלל העדכון שלנו להיות

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \bar{\mathcal{L}}(\theta_t)$$

יתרונות:

1. הרבה יותר מהיר לחישוב.

2. יכול לברוח ממנימום מקומי כי כל עדכון הוא "רועש".

חסרונות:

1. כל עדכון הוא "רועש" - אנחנו לא הולכים בכיוון הירידה הכי חדה בשביל כל הנתונים אלא בכיוון הירידה הכי חדה בשביל תת הקבוצה של הנתונים שבחרנו.

לכן שיטה יש את היתרונות והחסרונות שלה, לכן אין תשובה חד משמעית למה הכי כדאי להשתמש בו.

2 מטריקות הערכת המודל.

עכשיו אחרי שאימנו את המודל שלנו, איך אנחנו יודעים בפועל כמה טוב הוא? רק בגלל שדחפנו את הדאטה שלנו למודל ואמרנו תלמד לא אומר שהוא למד. אנחנו צריכים מדד כלשהו שיאמר לנו כמה טוב המודל מכליל את הלמידה שלו, כלומר אנחנו דרך כלשהי להעריך את המודל שלנו.

הרעיון הוא מאוד פשוט: במקום לדחוף את כל הדאטה שלנו למודל נחלק את הדאטה שלנו לשתי קבוצות זרות: **קבוצת אימון** (train set) ו**קבוצת מבחן** (test set). ככה נוכל למדוד את השגיאה של המודל על דאטה שהוא עוד לא ראה ושאנחנו יודעים מה הערך האמיתי שלו.

ניזכר כי

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (1)$$

מודד את המרחק בריבוע הממוצע של החיזויים (פרדיקציות) שלנו מהערכים האמיתיים, לכן אם נקטין את ה-MSE שלנו נקבל מודל טוב יותר. הבעיה היחידה היא שהיחידות מידה של MSE הן היחידות מידה של y בריבוע, לכן כדי לתקן את זה נגדיר

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (2)$$

כיוון שה-RMSE באותו יחידות מידה כמו של y, יהיה יותר קל לפרש אותו ולהחליט לפיו האם המודל טוב או לא טוב. עוד אפשרות היא להגדיר

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (3)$$

היתרון של MAE זה שהוא רגיש לאוטליירים (נקודות ששונות משמעותית משאר הדאטה) ולפיכך הוא בחירה טובה אם לדאטה שלי יש לעיתים ערכים קיצוניים.

עוד אפשרות נקראת R^2 והיא הדרך הכי שכיחה לסכם כמה טוב המודל שלי:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \quad (4)$$

כאשר $\bar{y} = \frac{1}{N} \sum_i y_i$ זה הממוצע של y.

האינטואיציה היא שהמונה מודד עד כמה רחוקות התחזיות מהאמת, בעוד שהמכנה מודד עד כמה הנתונים רחוקים מניבוי הממוצע בלבד.

אם המודל מושלם אז $R^2 = 1$, אם המודל מוצע (תרתי משמע חוזה כל הזמן את הממוצע) אז $R^2 = 1$, ואם $R^2 < 0$ אז המודל ממש גרוע.

דרך נוספת למדוד בעין כמה טוב המודל שלנו היא על ידי לשרטט את y מול $\hat{y}$: ככל שהמודל יותר טוב נצפה מהזוג $(y_i, \hat{y}_i)$ להיות קרוב יותר לישר $y = x$. לזה קוראים scatter plot.

ניתן גם לשרטט את השארית $Residual_i = y_i - \hat{y}_i$ וככל שהמודל יותר טוב נצפה שהשאריות יהיו קרובות ל-0.

הערה 1.2. חלוקה טובה של הדאטה ל-test \train היא 80%\20% בהתאמה, נרחיב על כך בהמשך.

תרגיל 2.2. פתחו את כלל האימון של אלגוריתם מורד הגרדיאנט לרגרסיה לינארית.

פתרון. ניזכר כי כלל העדכון הוא $\theta^+ = \theta - \eta \nabla \mathcal{L}(\theta)$ וכן ברגרסיה לינארית $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$. אם נציב $\hat{y}_i = w^T x_i + b$ נקבל

$$\mathcal{L}(w, b) = \frac{1}{N} \sum_{i=1}^N (y_i - (w^T x_i + b))^2$$

נגזור ביחס לכל אחד מהפרמטרים:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w} &= \frac{\partial}{\partial w} \frac{1}{N} \sum_{i=1}^N (y_i - (w^T x_i + b))^2 = \frac{2}{N} \sum_{i=1}^N -x_i (y_i - (w^T x_i + b)) = \frac{2}{N} \sum_{i=1}^N -x_i (y_i - \hat{y}_i) = \frac{2}{N} \sum_{i=1}^N x_i (\hat{y}_i - y_i) \\ \frac{\partial \mathcal{L}}{\partial b} &= \frac{\partial}{\partial b} \frac{1}{N} \sum_{i=1}^N (y_i - (w^T x_i + b))^2 = -\frac{2}{N} \sum_{i=1}^N (y_i - (w^T x_i + b)) = -\frac{2}{N} \sum_{i=1}^N (y_i - \hat{y}_i) = \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i)\end{aligned}$$

נהוג להגדיר $X = \begin{pmatrix} -x_1^T - \\ \vdots \\ -x_N^T - \end{pmatrix} \in \mathbb{R}^{N \times d}$ וכן $y = \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N$, $\hat{y} = \begin{pmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_N \end{pmatrix} \in \mathbb{R}^N$ ולכן לפי כפל מטריצות $\sum_{i=1}^N x_i (\hat{y}_i - y_i) = X^T (\hat{y} - y)$

כלומר

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w} &= \frac{2}{N} X^T (\hat{y} - y) \\ \frac{\partial \mathcal{L}}{\partial b} &= \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i)\end{aligned}$$

ולכן נקבל

$$\begin{aligned}w^+ &= w - \eta \frac{2}{N} X^T (\hat{y} - y) \\ b^+ &= b - \eta \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i)\end{aligned}$$

הערה 3.2. אם נסמן במקום $X = \begin{pmatrix} 1 & - & x_1^T & - \\ 1 & - & x_2^T & - \\ \vdots & & \vdots & \\ 1 & - & x_N^T & - \end{pmatrix}$, $y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$, $\beta = \begin{pmatrix} b \\ w_1 \\ \vdots \\ w_d \end{pmatrix}$ נוכל לכתוב

$$\beta^+ = \beta - \eta \frac{2}{N} X^T (\hat{y} - y)$$

3 רגרסיה פולינומיאלית.

נניח עכשיו במקום להניח שיש קשר לינארי בין הנתונים שלנו $\hat{y} = \beta_0 + \beta_1 x$, אנחנו חושדים שהנתונים שלנו לא מגיעים מקשר לינארי, אז אנחנו יכולים להרחיב את מודל הרגרסיה הלינארית להיות

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_d x^d$$

למודל כזה קוראים מודל רגרסיה פולינומיאלית.

אבל, השם הזה מטעה, כיוון שהקשר בין y לכל הפרמטרים של המודל הוא עדיין קשר לינארי.

איך מאמנים? בעיקרון אם בעבר ברגרסיה לינארית היה לנו $x = (x_1, x_2, \dots, x_d)$, אנחנו יודעים איך לאמן מודל מהצורה

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_d x_d$$

לכן פשוט נחשב $x_poly = (x, x^2, \dots, x^d)$ ופשוט יש לנו

$$\hat{y} = \sum_{n=1}^d x_poly_n \beta_n + \beta_0$$

לכן נוכל להתייחס לזה כתור מודל רגרסיה רגיל ולהשתמש בכל שיטות האימון שאנחנו כבר מכירים.

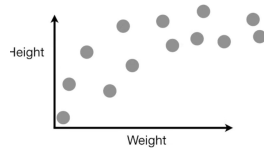
במידה ולכל דגימה x יש יותר מפיצ'ר אחד, כלומר $x = (x_1, \dots, x_p)$ ואנחנו רוצים את כל הפולינומים מדרגה לכל היותר d , אנחנו יכולים להגדיר

$$\hat{y} = \beta_0 + \sum_{1 \leq i \leq n} \beta_i x_i + \sum_{1 \leq i \leq j \leq n} \beta_{ij} x_i x_j + \sum_{1 \leq i \leq j \leq k \leq n} \beta_{ijk} x_i x_j x_k$$

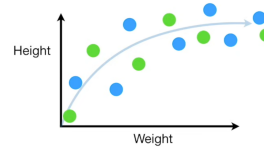
ושוב להתייחס למודל הזה כתור מודל רגרסיה לינארית.

4. BIAS VARIANCE TRADEOFF

נניח ואנחנו מודדים משקל וגובה של עכברים ומשרטטים את הדגימות שלנו בגרף



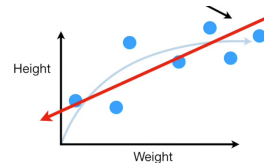
עכברים קלים הם נמוכים, עכברים כבדים הם גבוהים אבל בשלב מסוים עכבר נהיה שמן ולא גבוה יותר. אנחנו רוצים לחזות את גובה העכבר בהינתן המשקל שלו.



נניח ומשקל העכברים מגיע מכזה קשר, נרצה ללמוד אותו.

נחלק את הדאטה לקבוצת אימון וקבוצת מבחן, כאן הנקודות הכחולות יהיו האימון והירוקות יהיו המבחן.

אם ננסה להתאים קו לינארי בין הנקודות שלנו, ניתקל בבעיה, אין לו את "הגמישות" לתאר את היחס האמיתי בין המשקל לגובה (ה"קשת" שנוצרת בדאטה)



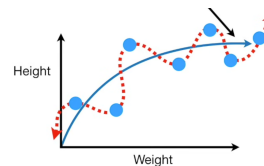
לא משנה איך נעביר את הקו הישר הוא בחיים לא יתעקס, כלומר הקו הישר בחיים לא ישחזר את היחס בין המשקל והגובה, לא משנה כמה טוב הוא על הקבוצת אימון שלנו.

חוסר היכולת של המודל שלנו לתפוס את היחס בין האמיתי של הפרמטרים שלנו נקרא $bias$.

בגלל שהקו הישר לא יכול להתעקס, יש לו הרבה $bias$ (מאנגלית - הטיה), כלומר הוא בקושי מתאר טוב את הדגימות.

לכן נאמר שהקו הישר עושה $underfit$ - לא תופס את הצורה של הדאטה שלנו.

מנגד, אנחנו יכולים להתאים פולינום ממעלה מאוד גדולה שיתאים בדיוק לכל הנקודות בקבוצת האימון שלנו.



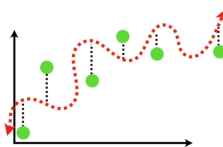
הפולינום הזה מתעקס באופן מעולה כמו שקבוצת האימון מתעקמת וממש מתאים את עצמו לקבוצת האימון.

בגלל שהפולינום הזה יכול להתעקס בצורה ממש טובה, יש לו $bias$ נמוך.

מבחינת MSE על ה- $train\ set$, הפולינום ינצח, כי יש לו $MSE = 0$.

עכשיו נעבור לקבוצת המבחן:

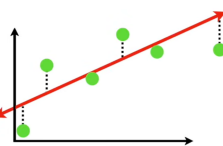
למרות שהפולינום עשה עבודה טובה בלמוד את קבוצת האימון, הוא ממש רחוק מלחזות את קבוצת המבחן



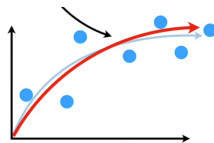
לכן יש לו שונות גבוהה מאוד שכן ההפרש בין הערך שנחזה לאיברים בקבוצת המבחן שונה ממש מהערך האמיתי שלהם.

במילים אחרות, קשה לחזות כמה הפולינום שלנו באמת למד. לפעמים הוא יהיה טוב, ולפעמים ממש גרוע.

מנגד, לקו הישר היה $bias$ מאוד גבוה, אבל יש לו פחות שונות כי הוא התאים את עצמו לצורה הכללית של הדאטה, ולא ספציפית לצורה של קבוצת האימון.

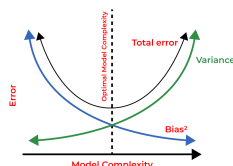


במילים אחרות, הקו הישר יביא תוצאות סבירות (ולא מעולות), אבל יהיה עקבי בתחזיות שלו. בגלל שהפולינום מתאים את עצמו לקבוצת האימון אבל לא לקבוצת המבחן, נאמר שהוא עושה *over fit*. אנחנו רוצים שהמודל האידיאלי שלנו יהיה בעל *bias* נמוך כלומר שיוכל לחקות את הקשר בין המשתנים שלנו בצורה סבירה, ושתהיה לו שונות נמוכה כלומר שיפיק תוצאות עקביות על דגימות שונות.



לדוגמה

אנחנו רוצים למצוא מודל שהוא בין *bias* נמוך לבין שונות נמוכה.



5. BIC.

כעת אנחנו נתקלים בבעיה: אנחנו מניחים שהדאטה שלנו לא לינארי, כלומר אנחנו צריכים לבחור איזשהו דרגה d שאנחנו מאמינים שתתאים לדאטה שלנו הכי טוב. הבעיה היא שדרגה נמוכה מדי תביא ל-*under fitting* ויותר מדי דרגה תביא ל-*over fitting*. אנחנו צריכים דרך לאזן בין איכות המודל לבין כמות הפרמטרים שבמודל.

לכן בדיוק יש לנו כלי בשביל למדוד איכות של מודל: נגדיר שבשביל מודל גרסיה עם n דגימות ו- k פרמטרים, נגדיר

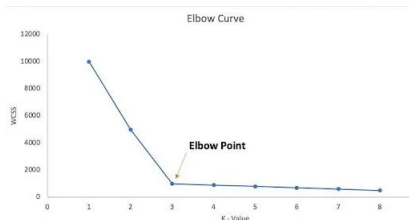
$$BIC = n \ln(MSE) + k \ln n$$

אנחנו רוצים BIC כמה שיותר נמוך, כלומר ההתאמה המירבית בין איכות המודל לגודל המודל.

אינטואיציה: $n \ln(MSE)$ מתגמל התאמה טובה של הדאטה (MSE נמוך $\Leftarrow BIC$ נמוך). $k \ln n$ נותן עונש למודלים מסובכים יותר (יותר פרמטרים $\Leftarrow BIC$ נמוך).

אם נעלה את הדרגה של הפולינום שאנחנו מתאימים, אולי נמצא התאמה יותר טובה אבל זה יעלה לנו את ה- BIC , אז אולי לא שווה להגדיל את המודל.

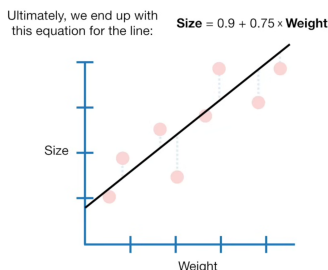
לכן, כדי למצוא את הפולינום הכי טוב, נאמן מודלים עם דרגות $d = 1, 2, 3, \dots, D$, נחשב BIC לכל מודל ונבחר את המודל עם ה- BIC הכי נמוך. הערה: אם ככל שאנחנו מגדילים את המודל ה- BIC לא עולה/ משתנה טיפה, נשתמש ב-*elbow - method* לבחור את d המתאים.



6 רגורזיציה.

6.1. RIDGE.

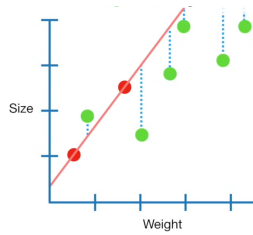
נניח ושוב אספנו מלא דאטה על משקל וגודל של עכברים



ואנחנו רוצים למדל את הקשר ביניהם. נאמן מודל לינארי ונגיע למשוואה

$$Size = 0.9 + 0.75 \cdot Weight$$

כשיש לנו הרבה דגימות אנחנו יודעים שהישר שנקבל מתאר את היחס בין גודל ומשקל. אבל מה אם היו לנו רק שתי דגימות ב-*train set*?



פה לדוגמה אם נאמן רק על שתי הנקודות האדומות נקבל את המשוואה $Size = 0.4 + 1.3 \cdot Weight$.

הרעיון מאחורי *ridge* הוא למצוא קו ישר שלא עושה את ההתאמה הכי טובה על ה-*train set*, אבל בתמורה יהיה לו הרבה פחות *variance* על ה-*test set*. במילים אחרות, אנחנו מעלים בכוונה את ה-*bias* של המודל בקצת כדי לקבל ירידה חדה ב-*variance*.

איך עושים? בעבר אם יש לי $\hat{y} = b + m_1x_1 + \dots + m_dx_d$ (עבור x עם d פיצ'רים), אז מצאנו $\arg \min_{b, m_1, \dots, m_d} \sum_{i=1}^N (y_i - \hat{y}_i)^2$, אבל עכשיו ננסה למצוא

$$\arg \min_{b, m_1, \dots, m_d} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^d m_j^2$$

כלומר אני מוסיף עונש למודל שלי ככל שיש לי יותר שיפוע, ו- λ קובע לי כמה חמור העונש הזה. מה הרעיון? ככל שיש לי שיפוע יותר גדול, ככה המודל שלי יותר רגיש לשינויים קטנים בדאטה, לכן כשאני ממזער גם את $\sum_{j=1}^d m_j^2$ אני מנסה למצוא את הפרמטרים לקו הישר שימדל את הדאטה

הכי טוב ויהיה כמה שפחות רגיש לשינויים. במקרה הזה נקבל את הישר $Size = 0.9 + 0.8 \cdot Weight$. עבור הישר שעושה *overfit* ועבור $\lambda = 1$ לדוגמה נקבל

$$\sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^d m_j^2 = 0 + 1 \cdot 1.3^2 = 1.69$$

והישר השני שקיבלנו יקבל

$$\sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^d m_j^2 = 0.3^2 + 0.1^2 + 1 \cdot 0.8^2 = 0.74$$

כלומר באמת קיבלנו התאמה טובה יותר.

הערה 1.6. λ יכול להיות כל ערך מ-0 ל- ∞ (לא כולל, כמובן). עבור $\lambda = 0$ נקבל פשוט MSE רגיל ועבור $\lambda \rightarrow \infty$ נקבל ישר שמקביל לציר ה- x . איך נחליט איזה ערך λ אנחנו רוצים? פשוט ננסה הרבה ערכים $\lambda_1, \lambda_2, \dots, \lambda_k$ ונשתמש ב-*k-fold cross validation* (נדבר בהמשך) כדי לקבוע איזה מהם הכי טוב לנו

הערה 2.6. הרבה פעמים *RIDGE* נקרא גם L^2 כי אם נסמן $m = (m_1, \dots, m_d)$ אז העונש שהוספנו $\lambda \sum_{j=1}^d m_j^2$ הוא $\lambda \|m\|_2^2$.

הערה 3.6. λ הוא היפרפרמטר, כלומר אני קובע לו ערך לפני שאני מאמן את המודל, וכאן אני לא מעדכן אותו תוך כדי האימון (בניגוד ל-*lr* שאני יכול להגדיל ולהקטין תוך כדי האימון).

6.2 LASSO

אותו קונספט כמו *RIDGE*, אבל במקום להעניש את המודל על ידי הוספה של $\lambda \|m\|_2^2$, נעניש אותו על ידי הוספת $\lambda \|m\|_1 = \sum_{j=1}^d |m_j|$.

מה ששונה פה זה שעבור λ ממש גדול אנחנו כן יכולים לקבל ישר שמקביל לציר ה- x , כלומר בניגוד ל- L^2 שיכול רק להקטין את הפרמטרים, L^1 יכול ממש לאפס אותם.

מתי זה שימושי? אם עכשיו אני רוצה לחזות גודל של עכברים כתלות במשקל שלהם, בגיל שלהם ובמה הטמפרטורה בחוץ. כפי שאפשר לנחש לטמפרטורה אין שום השפעה על גודל העכברים כי הם לא מתכווצים כשקר וגדלים כשחם, לכן היינו רוצים שבמהלך האימון המשקל שהמודל ישים על הטמפרטורה יהיה 0 ובכך יעלים את הפרמטר הטיפשי הזה. לכן יש לנו

$$Size = b + m_1 \cdot Weight + m_2 \cdot Age + m_3 \cdot Temp$$

ב-*LASSO* נפתור

$$\arg \min_{b, m_1, m_2, m_3} \sum_{i=1}^N (y_i - Size_i)^2 + \lambda (|m_1| + |m_2| + |m_3|)$$

ואז נגלה כי $m_3 = 0$ בעוד ש-*RIDGE* אף פעם לא יאפס אותו.

הערה 4.6. בעיה שמיד צורמת לנו זה ש- $|m|$ לא גזיר, אבל פה אנחנו יכולים להסכים להגדיר את הנגזרת של זה להיות

$$\frac{d}{dm}|m| = \text{sign}(m) = \begin{cases} 1 & m > 0 \\ -1 & m < 0 \\ 0 & m = 0 \end{cases}$$

הערה 5.6. גם כאן λ הוא היפרפרמטר, ולא תמיד הוא יהיה זהה לזה של ridge .

6.3 ELASTIC NET

אז L^2 עובד הכי טוב כשיש לי הרבה פרמטרים שימושיים בעוד ש- L^1 עובד הכי טוב כשיש לנו הרבה פרמטרים שאנחנו רוצים לאפס את ההשפעה שלהם בתחזית שלנו.

לכן אם אנחנו יודעים מה המודל שלנו מנסה לחזות ומה הפרמטרים שיש לנו אנחנו יודעים מה שימושי ומה לא, אבל מה אם יש לנו אוסף של פרמטרים גנריים שאנחנו לא יודעים עליהם כלום? איך נבחר L^1 או L^2 ?

אנחנו לא חייבים לבחור, אנחנו יכולים להשתמש ב- Elastic net ! מה זה עושה? כמו lasso ו- ridge , פשוט נפתור

$$\sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda_1 \|m\|_1 + \lambda_2 \|m\|_2^2$$

כדי למצוא את השילוב הכי טוב של (λ_1, λ_2) נשתמש ברוב ב- cross validation .

זה טוב בעיקר כשיש לנו קורולציה בין הפיצ'רים: אם לדוגמה x_1 ו- x_2 בעלי קורולציה חיובית, L^1 יבחר לאפס את אחד מהם (כי בעינינו להחזיק 2 עותקים של אותו פיצ'ר (בערך) זה לא יעיל) בעוד ש- L^2 פשוט יקטין את המשקל שלהם לבערך אותו דבר, ככה הם לא יתאפסו וכל אחד יוכל להשפיע בדרכו שלו על תחזיות המודל.

כשנשלב בין L^1 ו- L^2 , לא נאפס שום פרמטר, פשוט נקטין פרמטרים לא שימושיים.

6.4 EARLY STOPPING

נניח והפעם יש לנו המוני דגימות ואנחנו יכולים לאמן את המודל שלנו כמה שנרצה.

מצד אחד, אם נאמן את המודל יותר מדי, יש סכנה שנעשה overfitting כי נאמן אותו כל כך הרבה זמן שהוא יתחיל "לשנן" את ה- train set . מצד שני, אם אנחנו רוצים להימנע מזה ונאמן את המודל קצת מדי זמן, אנחנו מסתכנים ב- underfitting , כלומר המודל לא הספיק ללמוד את הקשר בין הפרמטרים שלנו וחוזר גיבריש.

איך נדע מתי הזמן האופטימלי לעצור את תהליך האימון של המודל?

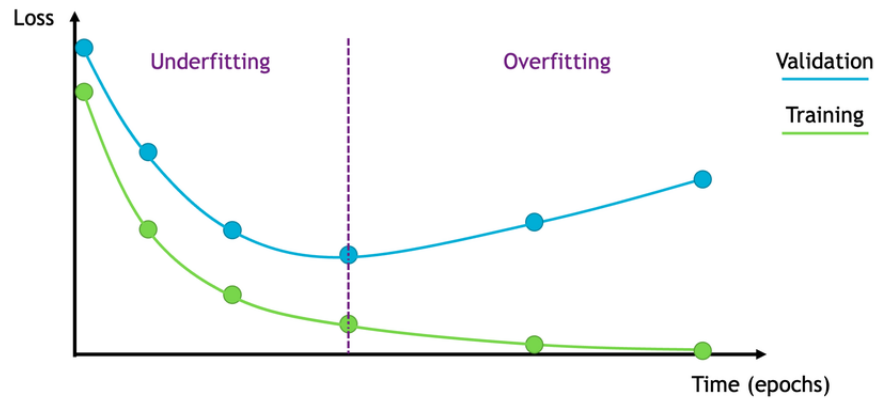
פשוט. נחלק את הדאטה שלנו ל- train set ו- validation set . נתחיל לאמן את המודל שלנו פעם אחרי פעם אחרי פעם על ה- train set שלנו, כל פעם שאנחנו מאמנים כל ה- train set נקרא epoch . אחרי כל epoch נמדוד את הביצועים של המודל על ה- validation set . בהתחלה ה- error שם ירד כי ככל שאנחנו מאמנים יותר זמן המודל לומד יותר טוב את הדאטה, אבל מתישהו ה- error יתחיל לעלות - שם אנחנו עושים overfit . היינו רוצים לעצור ברגע שאנחנו רואים שהמודל מתחיל לשנן אבל בשביל לדעת שהוא באמת עושה את זה אנחנו צריכים לחכות קצת לראות שאולי לא בטעות השגיאה עלתה אבל מיד אחרי זה ב- epoch הבא חזרה לרדת, לכן כשנעצור את המודל הוא יהיה לא טוב כי הוא כבר עשה שינוי, לכן אחרי כל epoch נשמור את הפרמטרים של המודל כל עוד ה- error שלו הכי קטן, ואז כשאנחנו עוצרים את הלמידה נחזיר את סט הפרמטרים האחרון ששמרנו. בפסאודו קוד זה נראה ככה:

```

for epoch in range(epochs) do
    if loss < best_loss then
        Save model parameters;
        best_loss ← loss;
    end
    if model is overfitting then
        return best saved parameters;
    end
end
end

```

ובזמן האימון ה- error יראה כך:



כמו שאנחנו מצפים, ה- $error$ של האימון תמיד יורד בעוד שה- $error$ על הדאטה החדש יורד עד לשלב מסוים, ואז מתחיל לעלות. לכן ניקח את הפרמטרים שיצאו לנו ב- $epoch$ הרביעי.

הערה 6.6. ב-2019 יצא מאמר בשם

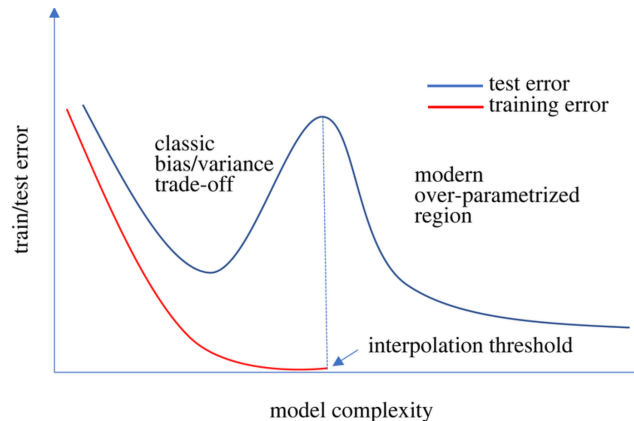
“DEEP DOUBLE DESCENT: WHERE BIGGER MODELS AND MORE DATA HURT” שמראה כי למודלים מסויימים, אם ממשיכים לאמן אותם, אז למעשה ה- $error$ יכול לרדת אחרי שהוא עולה, לפעמים מביא אפילו לשגיאה נמוכה יותר מאשר איפה שבעבר היינו מרימים ידיים ומכריזים שהמודל עושה $overfit$ ומפסיקים את האימון. המאמר אמר שיש 3 מקרים בהם זה יכול לקרות:

1. כשמגדילים את גודל המודל.

2. כשממשיכים לאמן את המודל כשהוא עושה $overfit$.

3. כשמגדילים את כמות הדאטה שנכנס למודל

בעיקרון מה שקורה זה שכל שאנחנו מגדילים כל אחד משלושת הגדלים האלו, המודל מתחיל בלמוד את הנתונים עד שהוא כבר מתאים את עצמו בדיוק לנתונים שנתנו לו, כלומר עושה $overfit$, אבל אם אני ממשיך להגדיל אז בשלב מסוים המודל יכול להתחיל ללמוד קשרים יותר “עמוקים” בדאטה, שמביאים לתוצאות יותר טובות.



6.5 סטנדרטיזציה.

אחרי שלמדנו על $l1/l2$ וכל שאר הרגולריזציות, עוד שיטה שמאוד מומלצת היא נירמול הדאטה: אנחנו רוצים להעביר את הדגימות שלנו x שיהיה להן ממוצע 0 ושונות 1, לכן נעביר

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad \text{ממוצע הדגימות ו-} \sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

עושים את זה כי יש אם כל הדאטה שלי ממש גדול, אם אני מזיז אותו שיהיה מרוכז סביב 0 יכול גם לחסוך לי בלשמור את כל הדגימות במחשב, וגם יהיה לי יותר קל ללמוד את ההתפלגות.

מיותר לציין שכאשר אנחנו משתמשים במודל, ננרמל את הדאטה לפי μ, σ שחישבנו על ה- $train set$.

מודלי סיווג.

7 רגרסיה לוגיסטית.

בשונה מרגרסיה לינארית/ פולינומיאלית שרוצות לחזות ערך רציף, כאן אנחנו מעוניינים לחזות האם משהו קורה או לא קורה. **הגדרה 1.7.** מודל קלסיפיקציה/ סיווג הוא מודל שמטרתו לנבא קטגוריה (*label*) מתוך קבוצה סופית של מחלקות. עד עכשיו היה לנו מודל מהצורה

$$\hat{y} = w^T x + b$$

ולכל דגימה x חזינו ערך $y \in \mathbb{R}$. בבעיית סיווג, יש לנו כמות סופית של מחלקות, $y \in \{c_1, c_2, \dots, c_k\}$, ולכל דגימה אני רוצה לחזות לאיזה מחלקה היא שייכת. לכן מודל רגרסיה לא יעבוד (גם כי מה זה אומר להיות בחלקה 6.5, וגם כי עבור ערך x ה- $\hat{y}$ שאנחנו חוזים יעבור את כל המחלקות?), ואנחנו נצטרך לפתח מודל אחר.

דוגמה 2.7. דוגמאות למודל קלסיפיקציה:

1. האם מייל הוא ספאם או לא ספאם.
 2. האם מטופל הוא חולה או בריא.
 3. האם תמונה היא של כלב, חתול, או ארנב.
 4. מי האדם שאנחנו רואים בתמונה.
 5. בהינתן ההתנהגות של האדם, לחזות איזה מזל הוא (טלה, שור, תאומים וכו').
- הרעיון: למודל יש הרבה מחשבות פנימיות ועמוקות, ובכללי הוא רוצה להחזיר את התשובה הכי הגיונית. במקרה שלנו, זה התיוג הכי סביר. לכן הוא מחשב את $\mathbb{P}(c_i|x)$ לכל מחלקה c_i ויחזיר את

$$\hat{c} = \arg \max_i \mathbb{P}(c_i|x)$$

כלומר המחלקה שהמודל יחזה היא זו שהכי הגיונית לתצפית. כשמאמנים את המסווג, המטרה היא למצוא פרמטרים θ שיגרמו לתצפיות להיות הכי סבירות.

כיום נעסוק בסיווג בינארי, כלומר כל קבוצת המחלקות שלי קורסת להיות $\{0, 1\}$. סה"כ אנחנו רוצים פונקציה

$$p_\theta(x) = \mathbb{P}(Y = 1|x; \theta)$$

שתחזה לנו מה הסיכוי לסיווג 1 בהינתן הדגימה (והפרמטרים של המודל).

יש לנו כמה דרישות מהמודל:

1. $0 < p_\theta(x) < 1$ לכל דגימה x .
2. p_θ צריכה להיות חלקה וקלה לאינטוס (יחסית).
3. התלות ב- x צריכה להיות לינארית, כלומר משקולת w_j חיובית תדחוף לי ערכי x גבוהים לכיוון $p_\theta(x) = 1$, ומשקולת w_j שלילית תדחוף לי ערכי x גבוהים לכיוון $p_\theta(x) = 0$.

הגדרה 3.7. בהינתן כל הדרישות הנ"ל, פונקציה אחת שבאה לראש הוא פונקציית הסיגמואיד

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

טענה 4.7. תכונות הסיגמואיד:

1. הטווח שלו הוא $\sigma : \mathbb{R} \rightarrow (0, 1)$.
 2. פונקציה חלקה (כלומר $\sigma \in C^\infty$).
 3. בהינתן $\sigma(z)$ עבור $z \in \mathbb{R}$, הנגזרת היא פשוט $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- לכן דרישות (1) ו-(2) מתקיימות, וכדי לקיים גם את דרישה (3) נגדיר את המודל שלנו להיות

$$p_\theta(x) = \sigma(w^T x + b)$$

הערה 5.7. כרגע $\mathbb{P}(Y = 1|x; \theta) = p_\theta(x)$, ולכן הסיכוי שהדגימה x תהיה במחלקה 0 הוא $\mathbb{P}(Y = 0|x; \theta) = 1 - p_\theta(x)$. למה סיגמואיד ולא פונקציה אחרת? סיגמואיד נראה ככה:



ספציפית לקחתי $y = \frac{1}{1+e^{-(6.4x-7)}}$. הרעיון הוא שאנחנו רוצים שככל ש- x קטן, שהסיכוי שלו להיות במחלקה 1 יקטן בהתאם.

לדוגמה, אם אני חוזה הצליח במבחן/ לא הצליח במבחן/ לא הצליח בשעות הלמידה שלו, הגיוני שהדגימות שבציר ה- x קרובות ל-0 (כלומר תלמיד למד ממש קצת) יהיו במחלקה 0 לעומת דגימות עם ערך x גבוה יותר שיהיו במחלקה 1 (כי תלמיד למד הרבה). כמובן גם שיכולים להיות תלמידים שלמדו קצת ועברו, ותלמידים שהשקיעו את חייהם בלמידה למבחן ונכשלו, ומה שטוב במודל הזה זה שהוא לא יושפע יותר מדי מהמיעוטים האלה. ככל שמישהו לומר יותר, ככה אנחנו מצפים שיצליח יותר. זה בדיוק מה שסיגמואיד אומר לנו, שככל ש- x גדול יותר (ו- w חיובי), נקבל שהערך y שלו (הסיכוי להיות במחלקה 1) גדול יותר.

שימו לב שאם אין לי הפרדה כלשהי בין המחלקות שלי (קורולציה נמוכה בין עובר/ נכשל כתלות בזמן הלמידה שלי) אז אין באמת סיגמואיד שיפריד לי בין המחלקות, לכן אנחנו מניחים שיש קורולציה כלשהי בין ה- x ל- y (כמו שהנחנו $y = w^T x + b$ ברגרסיה לינארית)

7.1 פונקציית loss.

להזכירכם, ברגרסיה לינארית $\hat{y} = w^T x + b$ היה לנו $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$ (ועוד $\lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2$ לרגולריזציה), אבל זה בשביל רגרסיה לינארית, פה אנחנו ברגרסיה לוגיסטית, לכן צריך לחשוב על מה יהיה ה- $loss$ שלנו. באופן כללי, כל מודל מנסה למקסם את הנראות (בלעז, *likelihood*) של התחזיות שלו:

$$L(\theta) = \mathbb{P}(y_1, y_2, \dots, y_N | x_1, x_2, \dots, x_N; \theta)$$

(כאשר x_i הדגימה שלנו, y_i המחלקה שאליה סיווגנו את x_i , ו- N כמות הדגימות שלנו).

אם אנחנו נניח שהדגימות i.i.d. (תמיד נכון אלא אם נאמר אחרת) נקבל

$$L(\theta) = \prod_{i=1}^N \mathbb{P}(y_i | x_i; \theta)$$

ניקח לוגריתם משני הצדדים ונקבל

$$\log L(\theta) = \sum_{i=1}^N \log \mathbb{P}(y_i | x_i; \theta)$$

וכיוון שפונקציית ה- $\log$ עולה, למקסם את הנראות שקול למקסם את לוג הנראות, כלומר אנחנו מחפשים

$$\hat{\theta} = \arg \max_{\theta} \log L(\theta)$$

הבעיה הנ"ל שקולה למזער את

$$\mathcal{L}(\theta) = - \sum_{i=1}^N \log \mathbb{P}(y_i | x_i; \theta)$$

עבור סיווג בינארי $y_i \in \{0, 1\}$ אפשר לכתוב $\mathbb{P}(y_i | x_i; \theta) = p_{\theta}(x_i)^{y_i} (1 - p_{\theta}(x_i))^{1-y_i}$ כי אם $y_i = 1$ אז נקבל $\mathbb{P}(y_i | x_i; \theta) = p_{\theta}(x_i)$ ואם $y_i = 0$ נקבל $\mathbb{P}(y_i | x_i; \theta) = 1 - p_{\theta}(x_i)$. מכאן נקבל

$$\mathcal{L}(\theta) = - \sum_{i=1}^N y_i \log p_{\theta}(x_i) + (1 - y_i) \log(1 - p_{\theta}(x_i))$$

הגדרה 6.7. אם נמצע את $\mathcal{L}$ נקבל את **הפסד האנטרופיה הצולבת הבינארית** (תרגום נואש מאנגלית - *Binary Cross - Entropy*)

$$BCE(\theta) = - \frac{1}{N} \sum_{i=1}^N y_i \log \sigma(w^T x + b) + (1 - y_i) \log(1 - \sigma(w^T x + b))$$

הערה 7.7. רק לוודא שאנחנו על אותו דף, $y_i \in \{0, 1\}$ (אפשר גם ± 1 אבל זה ידרוש קצת יותר חשיבה איך להגדיר את $\mathbb{P}(y_i | x_i; \theta)$ ככה), וכן $\sigma(w^T x + b) \in (0, 1)$.

7.2 איך מאמנים?

פשוט, עם אלגוריתם GD . נחשב נגזרות חלקיות:

נסמן:

$$1. z_i = w^T x_i + b$$

$$2. p_i = p_\theta(x_i) = \sigma(z_i)$$

$$3. \ell_i(\theta) = -(y_i \log p_i + (1 - y_i) \log(1 - p_i))$$

לכן ה- $loss$ שלנו הופך להיות

$$\mathcal{L}(\theta) = \sum_{i=1}^N \ell_i(\theta)$$

נרצה לחשב את $\frac{\partial \mathcal{L}}{\partial w}, \frac{\partial \mathcal{L}}{\partial b}$. מכלל השרשרת:

$$\frac{\partial \mathcal{L}}{\partial w} = \sum_{i=1}^N \frac{\partial \ell_i}{\partial p_i} \frac{\partial p_i}{\partial z_i} \frac{\partial z_i}{\partial w}$$

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^N \frac{\partial \ell_i}{\partial p_i} \frac{\partial p_i}{\partial z_i} \frac{\partial z_i}{\partial b}$$

נחשב:

$$\frac{\partial \ell_i}{\partial p_i} = -\frac{\partial}{\partial p_i} y_i \log p_i - \frac{\partial}{\partial p_i} (1 - y_i) \log(1 - p_i) = -\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i} = \frac{p_i - y_i}{p_i(1 - p_i)}$$

בנוסף,

$$\frac{\partial p_i}{\partial z_i} = \frac{\partial}{\partial z_i} \sigma(z_i) = \sigma(z_i)(1 - \sigma(z_i)) = p_i(1 - p_i)$$

וכן

$$\begin{aligned} \frac{\partial z_i}{\partial w} &= \frac{\partial}{\partial w} (w^T x_i + b) = x_i \\ \frac{\partial z_i}{\partial b} &= \frac{\partial}{\partial w} (w^T x_i + b) = 1 \end{aligned}$$

לכן סה"כ נקבל

$$\begin{aligned} \frac{\partial \ell_i}{\partial w} &= \frac{p_i - y_i}{p_i(1 - p_i)} \cdot p_i(1 - p_i) \cdot x_i = (p_i - y_i)x_i \\ \frac{\partial \ell_i}{\partial w} &= \frac{p_i - y_i}{p_i(1 - p_i)} \cdot p_i(1 - p_i) \cdot 1 = p_i - y_i \end{aligned}$$

כלומר

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w} &= \sum_{i=1}^N (p_i - y_i)x_i \\ \frac{\partial \mathcal{L}}{\partial b} &= \sum_{i=1}^N p_i - y_i \end{aligned}$$

ומכאן כלל העדכון לפי האלגוריתם יהיה

$$\begin{aligned} w &\leftarrow w - \eta \sum_{i=1}^N (p_i - y_i)x_i \\ b &\leftarrow b - \eta \sum_{i=1}^N p_i - y_i \end{aligned}$$

7.3 גבול ההחלטה.

גבול ההחלטה הוא כלל שלפיו אנחנו מחליטים מה יהיה הסיכוי של הדגימה לשנו.

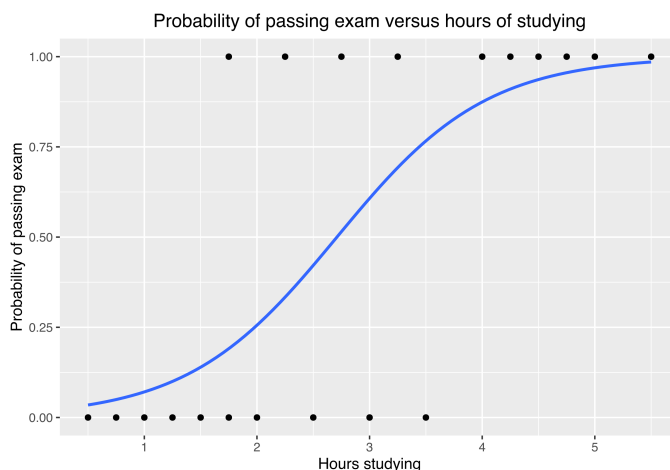
על פניו ניכר כי כלל ההחלטה הוא מאוד פשוט: יש לנו מודל

$$p(x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

שאומר לנו לכל דגימה x_i מה הסיכוי שהמחלקה שלה $y_i = 1$. בגישה הכי נאיבית, אם $p(x) \geq 0.5$, נסווג את הדגימה להיות 1, אחרת נסווג להיות 0. פרקטית, נקבל

$$y = 1 \iff p(x) \geq 0.5 \iff \frac{1}{1 + e^{-(w^T x + b)}} \geq 0.5 \iff w^T x + b \geq 0$$

גבול ההחלטה אצלנו הוא סקלר במקרה ש- x הוא סקלר, אבל במודלים אחרים הוא יכול להשתנות (לדוגמה ב-SVM הוא היפרמישור (נגיע בהמשך)).



אם לדוגמה אנחנו רוצים לסווג כ-1 רק דגימות שאנחנו ממש בטוחים לגביהן, אז לפי התמונה פה ניקח את גבול ההחלטה שלנו להיות 0.2 ואז הפעם נקבל

$$y = 1 \iff p(x) \geq 0.2 \iff \frac{1}{1 + e^{-(w^T x + b)}} \geq 0.2 \iff w^T x + b \geq -\ln 4$$

ובאופן כללי אם רוצים שגבול ההחלטה יהיה p אזי נקבל

$$y = 1 \iff \dots \iff w^T x + b \geq \ln \frac{p}{1-p}$$

כלומר בגדול רגרסיה לוגיסטית היא רגרסיה לינארית על ה- $\log \text{ odds}$.

הערה 8.7. לגבול ההחלטה אנחנו קוראים גם threshold . יותר פרטים יבואו בפרק על הערכת המודל המסווג.

ככל שנגדיל את כמות המימדים ככה גם מימד גבול ההחלטה יגדל. זה נכון כי נדוגמה אם ניקח $x = (x_1, x_2, x_3)$ אזי המודל שלנו יהיה

$$p(x) = \frac{1}{1 + e^{-(w_1 x_1 + w_2 x_2 + w_3 x_3 + b)}}$$

ואם נקבע $\text{threshold} = p$ נקבל

$$y = 1 \iff \dots \iff w_1 x_1 + w_2 x_2 + w_3 x_3 + b \geq \ln \frac{p}{1-p}$$

כלומר נקבל שבעצם אנחנו רוצים לאמן "מישור מפריד" שיחלק לנו את שתי המחלקות. במימד גבוה הרעיון ישאר אותו דבר, אם יהיה לנו $x = (x_1, \dots, x_d)$ אז נרצה למצוא מישור $w^T x + b = 0$ שיפריד לנו בין שתי המחלקות באופן הכי טוב.

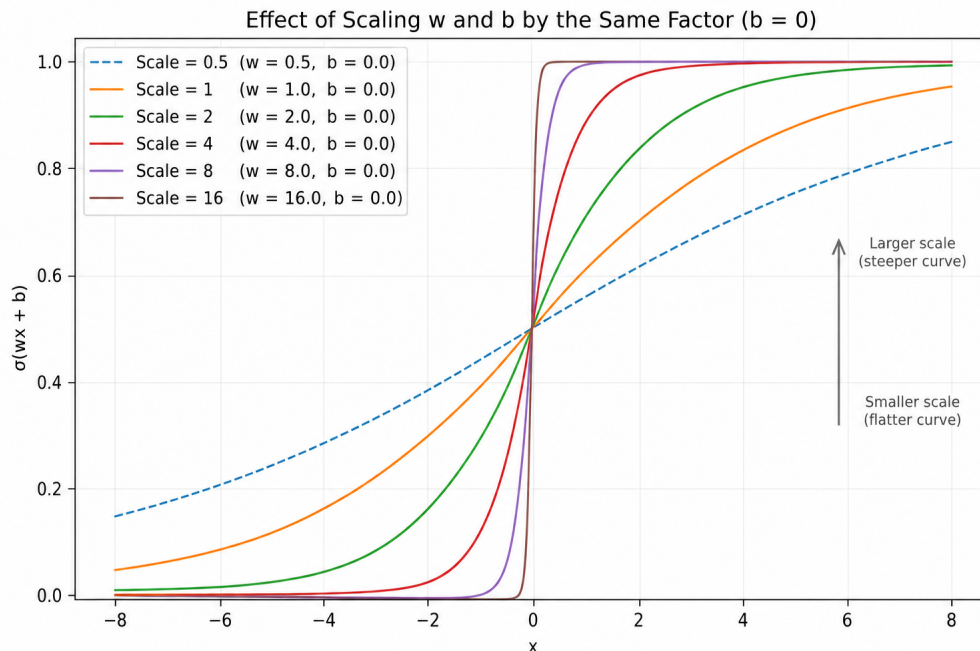
7.4 רגולריזציה.

אחרי שאימנו את המודל שלנו לפי

$$w \leftarrow w - \eta \sum_{i=1}^N (p_i - y_i) x_i$$

$$b \leftarrow b - \eta \sum_{i=1}^N p_i - y_i$$

אנחנו נשים לב לתופעה משונה, הפרמטרים θ שלנו נורא גדולים. למה בעצם זה קורה? אם θ זה הסט פרמטרים שמפריד לי את שתי המחלקות בצורה הכי טובה, אם ניקח $\theta' = 2\theta$ נקבל $\mathcal{L}$ נמוך יותר וזה הגיוני, כי ככל שהערכים של w, b גבוהים יותר, ככה הסיגמואיד שלנו יראה יותר ויותר כמו פונקציית מדרגה (כי $\sigma(\theta \cdot x) \xrightarrow{\theta \rightarrow \infty} \mathbb{I}_{x>0}(x)$), כמתואר בתרשים

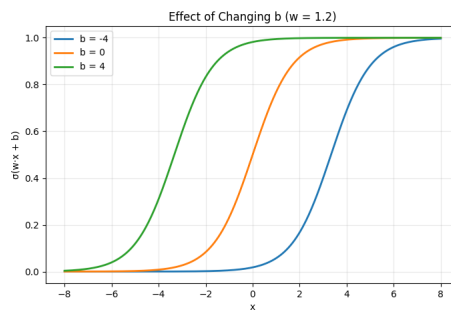


מה למדנו לעשות כאשר אנחנו לא רוצים שהפרמטרים של המודל ישתוללו יותר מדי? נוכל לשים רגולריזציה! לפיכך ה- $loss$ שלנו יראה

$$\mathcal{L}(w, b) = \text{BCE}(w, b) + \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2$$

ושב המקשולות שלנו לא ירצו לגדול פי $1e10$ לדוגמה בשביל לסחוט עוד שיפור של $1e-3$ ב- $loss$.

הערה 9.7. לכאורה זה נראה שגם w וגם b הם אלו שגורמים לסיגמואיד להתפוצץ ולהפוך לפונקציית מדרגה, אבל בתכלס כל מה ש- b עושה זה מזיז לי את הסיגמואיד ימינה/שמאלה



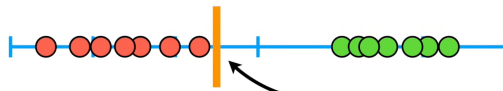
לכן גם עכשיו הוא לא צריך להיכלל ברגולריזציה.

8 SVM (מכונת וקטורים תומכים).

אנחנו ממשיכים עם סיווג בינארי של הדאטה שלנו. נניח הפעם אנחנו מדדנו עוד פעם את המשקל של עכברים כדי לסווג אותם כ"שמנים"/"לא שמנים" (בלי להעליב אף אחד) וקיבלנו את זה:

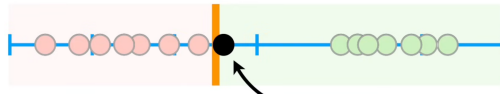


כשהנקודות האדומות הן עכברים לא שמנים והנקודות הירוקות הם העכברים השמנים. בהתבסס על הדגימות האלה אני יכול לקבוע ערך סף (*threshold*)

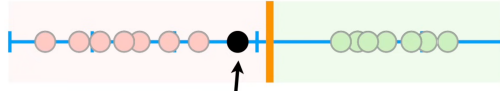


ככה שאם נדגום עכבר חדש עם פחות משקל מאשר הערך סף, נסווג אותו כ"תור לא שמן", וכשנשקול עכבר חדש עם יותר משקל מהערך סף, נסווג שהוא כן שמן.

אבל מה יקרה אם נדגום עכבר כזה:



כיוון שהמשקל שלו יותר גבוה מהערך סף, נסווג אותו כתור עכבר שלם, אבל זה לא הגיוני כל כך כי הוא הרבה יותר קרוב לעכברים הלא שמנים מאשר לעכברים השמנים, לכן המסקנה היחידה היא שהערך סף הזה גרוע. איך אנחנו יכולים להשתפר? אם נחזור לכל הדגימות שלנו, אנחנו יכולים להתמקד בדגימות שבקצה של כל אשכול ולהשתמש באמצע הקטע שהן יוצרות כתור ערך סף.



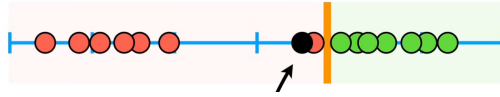
עכשיו כשנדגום את אותו עכבר ממקודם, המשקל שלו יהיה יותר קרוב לאלו שלא שמנים מאשר לאלו שמנים, כלומר נסווג אותו כתור לא שמן. **הגדרה 1.8.** המרחק הכי קצר בין הדגימות וה-*threshold* נקרא **שול** (*shul*, באנגלית - *margin*) (יש שול אחד כי עושים מינימום על מרחק שעל כל הדאטהסט, לא על קבוצה 0 בנפרד וקבוצה 1 בנפרד).

כיוון ששמנו את ה-*threshold* באמצע של שתי הדגימות "הכי קרובות", יוצא הוא המרחק של ה-*threshold* משתי נקודות. גם יוצא שבגלל שה-*threshold* בדיק באמצע של שתי הנקודות האלו, השול הזה הכי גדול שהוא יכול להיות. אם לדוגמה נזיז את ה-*threshold* שמאלה, אז המרחק בינו לבין דגימה של עכבר לא שמן תהיה נמוכה יותר, כלומר השול יהיה נמוך יותר משהיה.

הגדרה 2.8. מסווג שנותן לנו את השול הכי גדול נקרא (למרבית ההפתעה) **מסווג השול המקסימלי** (*maximum margin classifier*). **שאלה 3.8.** אבל מה עושים אם עכשיו היו לי דגימות כאלו:



זה אותם עכברים ממקודם פשוט שעכשיו הוספתי עוד עכבר שבטעות סיווגתי אותו להיות לא שמן כשבפועל הוא שמן. אם הייתי רוצה להתאים מסווג שול מקסימלי, אז ה-*threshold* באמת היה באמצע הדרך בין העכבר הלא שמן עם המשקל הגבוה ביותר ובין העכבר השמן עם המשקל הנמוך ביותר. הוא יהיה ממש קרוב לעכברים השמנים, אבל ממש רחוק מהעכברים הלא שמנים, הכל בגלל שהוספתי *outlier* אחד בטעות. זה בעיה כי אם עכשיו יש לי עכבר חדש



אז באופן אוטומטי אני אסווג אותו כתור עכבר לא שמן, למרות שהוא קרוב יותר חקבוצת העכברים השמנים מאשר לאלו שלא שמנים. זה הופך את מודל מסווג השול המקסימלי למודל עלוב, כי הוא ממש רגיש ל-*outliers* ואני רוצה מודל שיהיה די אדיש אליהם (כי טעויות קורות ואני לא רוצה מודל שיהפוך לי לגרוע גם אם עשיתי טעות אחת על דאטה גדול).

כדי לשפר את המודל שלנו אנחנו חייבים לבחור *threshold* שיהיה אדיש ל-*outliers*, כלומר אנחנו חייבים להרשות שאולי בדאטה שלנו יש סיווגים לא נכונים. לדוגמה, אם נשים את ה-*threshold* בין הנקודה האדומה השנייה הכי ימנית לבין הנקודה הירוקה הכי שמאלית, נקבל שאכן יש לנו דגימה שסיווגנו לא נכון, אבל מצד שני המודל שלנו עכשיו מסווג באופן יותר הגיוני את אותה נקודה ממקודם, כי היא באמת קרובה יותר לעכברים השמנים, יותר מאשר העכברים הלא שמנים.

הגדרה 4.8. הפרדה לקבוצות שלא מרשה טעויות בסיווג נקראת **הפרדה קשיחה**. הפרדה לקבוצות שכן מרשה טעויות בסיווג נקראת **הפרדה רכה**.

שאלה 5.8. איך נסכם את זה במונחי $bias \setminus variance$?

- בסיווג קשיח, בחרנו *threshold* שרגיש לשגיאות ב-*training data* שלנו, כלומר היה לו *bias* נמוך (כי הוא לופד את השגיאות שלנו על ה-*train*), אבל היה לו *variance* גבוה כי בתפורה היה לו הרבה טעויות על דגימות חדשות, כלומר עשינו *over fit*.
- בסיווג רך, בחרנו *threshold* שפחות היה רגיש ל-*outliers* ב-*train* שלנו וכתוצאה מכך סיווג לא נכון חלק מהנקודות שלנו, היה לו *bias* נמוך (כי הוא למד פחות טוב על ה-*train* שלנו). מצד שני, כשהבאנו לו דגימות חדשות הוא כן ביווג אותן יותר טוב ממוקדם, כלומר היה לו *variance* נמוך יותר.

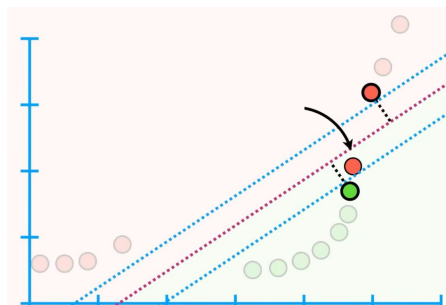
הגדרה 6.8. שול שמתקבל כתוצאה מהפרדה רכה נקרא **שול רך**.

במרחק של שול רך מן ה-*threshold* שלנו אנחנו יכולים שיהיו לנו נקודות מהקבוצה שלנו, ואנחנו יכולים שיהיו לנו *outliers*. איך אנחנו יודעים כמה *outliers* אני מוכן לקבל? את זה נעשה בעזרת *cross - validation*, עוד פרטים בהמשך.

הגדרה 7.8. מסווג שמשתמש בשול רק נקרא **מסווג שול רך** (באנגלית - *soft margin classifier*).

הגדרה 8.8. דגימות הנמצאות בתוך השול הרך נקראות **תומכים** (תומכים במה? מי יודע!).

אם עכשיו לכל עכבר אני מודד משקל וגובה, אני אקבל משהו כזה:



כאשר ציר ה- x הוא משקל וציר ה- y הוא גובה העכבר.

נאמן מסווג שול רך ונקבל את השול הרך האדום (אם אני מעליי אני לא שמן ואם אני מתחתיו אני כן שמן). השול הרך נמדד ביחס לשתי הנקודות המודגשות. הקווים הכחולים נותנים לנו מובן של כל הנקודות האחרות ביחס לשול הרך. כמו מקודם נשתמש ב- $cross-validation$ ונקבל שלאפשר לנקודה האדומה שמתחת לשול להיות $outlier$ יתן לנו סה"כ מודל טוב יותר.

הגדרה 9.8. מודל ה- SVM הוא מסווג לינארי מהצורה

$$f_{\theta}(x) = w^T x + b$$

שחזרה

$$\hat{y}_i = \text{sign}(w^T x_i + b)$$

כך שהמישור $w^T x + b = 0$ יביא את השול הכי גדול.

8.1 פונקציית loss.

נפרמל את הבעיה באופן מתמטי:

נניח שהדאטה שלנו פריד לינארית, אנחנו פשוט צריכים לבחור איזה ישר מפריד אנחנו רוצים. נסמן את קבוצת המחלקות להיות הפעם $\{-1, 1\}$ (יותר נוח למתמטיקה שתבוא), לכן אנחנו יכולים להניח שקיימים w, b כך שלכל דגימה (x, y) :

$$y(w^T x + b) > 0$$

כלומר אם $y = 1$ אז x מעל המישור המפריד, ואם $y = -1$ אז x מתחת למישור המפריד.

כאמור, יש ∞ מישורים מפרידים, אבל אנחנו רוצים לבחור את האחד הכי חזק, כלומר אנחנו רוצים את השול הכי גדול. נזכור כי המרחק מנקודה x למישור $w^T x + b = 0$ הוא

$$\frac{|w^T x + b|}{\|w\|}$$

(הנורמה היא נורמה אוקלידית כמובן, כלומר $\|\cdot\|_2$).

נחפש w, b שיקיימו לשני התומכים

$$y_i(w^T x_i + b) = 1$$

ולכן כל נקודה אחרת תביא לנו

$$y_i(w^T x_i + b) \geq 1$$

(כי התומכים הם הנקודות הכי קרובות לישר שלנו (אין $outliers$ להזכירם כי הנחנו שהדאטה פריד לינארית). לפיכך השול הוא פשוט $\frac{1}{\|w\|}$ אז למקסם את השול זה שקול ללמזער את $\|w\|$. נכתוב כבעיית אופטימיזציה:

$$\begin{aligned} \min_{w, b} \frac{1}{2} \|w\|^2 \\ \text{s.t. } y_i(w^T x_i + b) \geq 1, \forall i \end{aligned}$$

זה המודל עבור ההפרדה הקשיחה.

הבעיה היא שלפעמים הדאטה שלנו לא פריד לינארית. נסמן $\xi_i \geq 0$ להיות כמה הדגימה ה- i רחוקה מהשול, לכן נוכל לכתוב את הבעיה כתור

$$\begin{aligned} \min_{w, b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \\ \text{s.t. } y_i(w^T x_i + b) \geq 1 - \xi_i \\ \xi_i \geq 0 \end{aligned}$$

זה הופך לנו את הבעיה מהפרדה קשה להפרדה רכה. נשים לב כי אם באמת $y_i(w^T x_i + b) \geq 1$ אז $\xi_i = 0$ כי הנקודה מחוץ לשול, אם $0 < y_i(w^T x_i + b) < 1$ אז גם $0 < \xi_i < 1$ כי הנקודה בתוך השול, ואם $y_i(w^T x_i + b) \leq 0$ אז הנקודה היא $outlier$ ו- $\xi_i \geq 1$.

C כאן הוא כמו λ ברגולריזציה, C גבוה יעניש בחוזקה כל טעות והמודל יתאמץ הרבה כדי להקטין דווקא את השוליים - יכול לגרום ל- $overfit$. C קטן פחות ישנה לו טעויות פה ושם לכן המודל יעבוד פחות קשה בלהקטין את השוליים כדי להתאים את עצמו לדגימות העכשוויות - פחות $overfit$.

בדרך כלל אני עדיין ארצה להעניש את המודל שלי על ξ_i גדולים מדי, לכן נשתמש ב- $C = 1$.

אם נחפש נוסחא ל- ξ_i נקבל

$$\xi_i = \max(0, 1 - y_i(w^T x_i + b))$$

$$\begin{aligned}\xi_i = \max(0, 1 - y_i(w^T x_i + b)) &\Rightarrow \xi_i \geq 1 - y_i(w^T x_i + b) \Rightarrow y_i(w^T x_i + b) \geq 1 - \xi_i \\ &\Rightarrow \xi_i \geq 0\end{aligned}$$

לכן נציב בחזרה בבעיית אופטימיזציה שלנו ונקבל שאנחנו מחפשים

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \max(0, 1 - y_i(w^T x_i + b))$$

זה ממש דומה כבר לפונקציית $loss$, לכן נגדיר:

הגדרה 10.8. נגדיר $hinge - loss$ של בעיית ההפרדה הרכה להיות

$$Hinge(\theta) = \sum_{i=1}^N \max(0, 1 - y_i(w^T x_i + b))$$

לכן המודל שלנו יחשב תמיד (וימזער)

$$\mathcal{L}(\theta) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \max(0, 1 - y_i(w^T x_i + b))$$

שאלה 11.8. למה אין לנו פקטור של $\frac{1}{N}$ מחוץ לסכום?

תשובה: אפשר לשים, היסטורית זה לא מה שעשו. אנחנו לא מסתכלים פה על הטעות הממוצעת, אנחנו סוכמים את ξ_i כדי להיפטר מהאילוצים. לטריק הזה קוראים "צורת אפיגרף". גם אם נשים $\frac{1}{N}$ לפני הסכום, C יאבד מכוחו ואז כדי שתהיה לו אותה עוצמה נצטרך להציב $C' = CN$.

שאלה 12.8. למה אנחנו תקועים עם פקטור של $\frac{1}{2}$ ולא נותנים לו להיות λ כללי?

תשובה: אנחנו לא תקועים איתן, בהתחלה הבעיה שלנו הייתה

$$\begin{aligned}\min_{w,b} \frac{1}{2} \|w\|^2 \\ s.t. \ y_i(w^T x_i + b) \geq 1, \forall i\end{aligned}$$

ואותו פקטור $\frac{1}{2}$ נשאר איתנו עד הסוף, לכן אפשרי לכתוב

$$\mathcal{L}(\theta) = \lambda \|w\|^2 + C \sum_{i=1}^N \max(0, 1 - y_i(w^T x_i + b))$$

פשוט יהיו לנו 2 היפרפרמטרים. היסטורית לפני האופטימונה שמו את הפקטור $\frac{1}{2}$ כדי שבגזירה יהיה משהו יפה $w = \frac{\partial}{\partial w} \frac{1}{2} \|w\|^2$, אבל באמת שאין לו חשיבות.

שאלה 13.8. האם אנחנו יכולים להוסיף $\lambda_1 \|w\|_1$ לפונקציית $loss$?

תשובה: כן, כמו ברגוריאזיה רגילה, פשוט שאז זה לא יהיה $loss$ של SVM , ואם אנחנו מוסיפים את זה אז אי אפשר להפעיל את "תעלול הגרעין" (שנדבר עליו בהמשך).

שאלה 14.8. למה דווקא לסכום יש פקטור של C ולא $\|w\|$ כמו ברגוריאזיה רגילה?

תשובה: כי ברגוריאזיה, הדבקנו היפרפרמטר על כל דבר שיכל "להתפוצץ ל- ∞ ", ופה כל ξ_i יכולים בהחלט להתפוצץ ל- ∞ אם נתאים מסווג גרוע.

שאלה 15.8. איך $\mathcal{L}(\theta) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \max(0, 1 - y_i(w^T x_i + b))$ הזה קשור ל- $\mathcal{L}(\theta) = - \sum_{i=1}^N \log \mathbb{P}(y_i | x_i; \theta)$ שהעגנו בחלק הקודם? הרי

אמרנו שזה מה שכל מודל רוצה למזער?

תשובה: הם לא קשורים. מודל רגרסיה מניח שהנקודות שלי באות מהתפלגות כלשהן, ולכן רוצה ללמוד אותן, אבל SVM הוא לא מודל הסתברותי, הוא מודל גאומטרי, שבא מהתבוננות בנקודות והרצון למצוא להם ישר מפרד עם השוליים הכי גדולים.

8.2 איך מאמנים?

כמובן שאנחנו יכולים להשתמש באלגוריתם מורד הגרדיאנט (כי הכל קמור ושאר התנאים שתראו בקורס "אופטימיזציה" מתקיימים), לכן נעשה כמו שעשינו ברגרסיה לוגיסטית ונסמן:

$$\begin{aligned}m_i &= y_i(w^T x_i + b) \\ \ell_i &= \max(0, 1 - m_i)\end{aligned}$$

$$\mathcal{L}(w, b) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \ell_i$$

ונשים לב כי את הסכום $\sum_{i=1}^N$ אפשר לכתוב גם כתור $\sum_{i \in I}$ כאשר $I = \{i : m_i < 1\}$ (כי אחרת $\ell_i = 0$), לכן עבור $i \in I$ יתקיים $\ell_i = 1 - m_i$

$$\begin{aligned} \frac{\partial}{\partial w} \frac{1}{2} \|w\|^2 &= w \\ \frac{\partial}{\partial w} \ell_i &= \frac{\partial}{\partial w} (1 - y_i (w^T x_i + b)) = -y_i x_i \end{aligned}$$

ומכאן

$$\nabla_w \mathcal{L} = w - C \sum_{i \in I} y_i x_i$$

ולפי b נקבל

$$\frac{\partial}{\partial w} \ell_i = -y_i$$

כלומר

$$\nabla_w \mathcal{L} = -C \sum_{i \in I} y_i$$

ומכאן שכללי העדכון יהיו

$$\begin{aligned} w &\leftarrow (1 - \alpha)w + \alpha C \sum_{i \in I} y_i x_i \\ b &\leftarrow b + \alpha C \sum_{i \in I} y_i \end{aligned}$$

* אם $m_i = 1$ אז ניקח תת-גרדיאנט כלשהו, זה לא משנה כי מדובר בקבוצה ממידה 0.

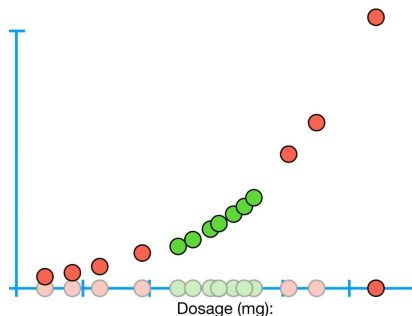
9 Kernel-SVM - כשהדאטה לא פריד לינארית.

באופן כללי אנחנו יכולים להסתפק פה במודל די סביר שמאפשר להתמודד עם $outliers$, אבל מה אם אני רוצה לסווג את הדאטה הבא:



שמתאר לי פציינטים שנרפאו ממחלה כתלות בכמה תרופה הבאתי להם? אם המינון נמוך מדי שום דבר לא יקרה, אם המינון רב מדי הם ימותו (או לא יבראו מסיבה פחות קיצונית), לכן צריך מינון מתאים כדי לסווג שחולה הבריא. לא משנה איפה נשים את ה- $threshold$ שלנו, אף פעם לא יהיה לנו סיווג טוב כי הדאטה שלנו לא פריד לינארית! בשביל זה המציאו את המודל הבא:

אינטואיציה: בואו ניקח את החולים שלי שנתתי להם מינון של x מיליגרם מהתרופה שלי ונפרוש אותם על ציר ה- x שלי, ולכל חולה נחשב את המינון בריבוע x^2 של התרופה שהבאתי לו, ונשים את זה על ציר ה- y .



והפעם אני יכול להפריד את הדאטה שלי לינארית הרבה יותר בקלות!

נעביר קו מפריד ונסווג דגימה חדשה d לפי האם d^2 (הקואורדינטה ה"חדשה" שלה בציר ה- y) מעל הקו המפריד או לא.

הצעה 1.9. מכאן שהרעיון המרכזי מאחורי מודל ה- SVM הוא לינארי הוא:

1. להתחיל עם דאטה בממד נמוך (יחסית), ששם אי אפשר להפריד אותו לינארית.

2. להעביר את הדאטה למימד יותר גבוה (בצורה חכמה!).

3. למצוא פסווג לינארי שמפריד את הדאטה במימד הגבוה.

נניח שאנחנו רוצים להעביר את כל הדאטה שלנו לפי הכלל $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$, כאשר D יכול להיות גדול מאוד ביחד ל- d , אולי אפילו אינסופי. אזי נאמן את ה- SVM במימד החדש להיות

$$f_\theta(x) = w^T \phi(x) + b$$

והיופי פה זה שאם בחרתי את ϕ טוב, עכשיו אני באמת יכול להפריד את הדאטה שלי לפי $\hat{y}_i = f_\theta(x_i)$.

מה הבעיה פה?

כדי לחשב את $w^T \phi(x)$ אנחנו צריכים:

1. לחשב את $\phi(x)$ מלכתחילה.

2. לאכסן את $\phi(x)$ ואת w במחשב שלנו.

הבעיה היא שאם D שלי גדול, זה ממש בעייתי לי לשמור את שני הוקטורים הללו בזיכרון, וזה גם בעייתי לי להכפיל אותם כל הזמן כי זה נורא יקר חישובית.

לכן אנחנו צריכים לשכתב את המודל שלנו בצורה יותר חכמה.

אנחנו רוצים לפתור

$$\begin{aligned} \min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \\ \text{s.t. } y_i(w^T \phi(x_i) + b) \geq 1 - \xi_i, \forall i \\ \xi_i \geq 0 \end{aligned}$$

לפי שיטת כופלי לגראנז' נגדיר את הלגראנז'אן להיות

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \alpha_i (y_i(w^T \phi(x_i) + b) - 1 + \xi_i) - \sum_{i=1}^N \mu_i \xi_i$$

ורוצים למזער לפי w . לכן

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_{i=1}^N \alpha_i y_i \phi(x_i)$$

כלומר

$$w = \sum_{i=1}^N \alpha_i y_i \phi(x_i)$$

ומקבלים עוד דברים...

לכן נכתוב

$$\begin{aligned} f_\theta(x) = w^T x + b &= \left(\sum_{i=1}^N \alpha_i y_i \phi(x_i) \right)^T \phi(x) + b \\ &= \sum_{i=1}^N \alpha_i y_i \langle \phi(x_i), \phi(x) \rangle + b \end{aligned}$$

הגדרה 2.9. נגדיר את פונקציית הגרעין להיות $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$

כלומר פונקציית הגרעין מחזירה את המכפלה הפנימית בין ϕ של x_i, x_j , אבל כפונקציה של x_i, x_j . לכן נוכל להמשיך לכתוב

$$f_\theta(x) = \sum_{i=1}^N \alpha_i y_i K(x_i, x) + b$$

והמסווג יהיה

$$\hat{y}_i = \text{sign} \left(\sum_{i=1}^N \alpha_i y_i K(x_i, x) + b \right)$$

שאלה 3.9. למה זה יותר טוב מאשר $\hat{y}_i = \text{sign}(w^T \phi(x_i) + b)$?

1. אנחנו לא מחשבים $\phi(x)$.
 2. לא שופרים את w בזיכרון.
 3. לעולם לא עובדים ב- $\mathbb{R}^D$.
 4. הפסווג תלוי בגרעין בלבד.
 5. גבול ההחלטה הוא לא לינארי בפרחב הפקורי.
- גרענים שימושיים:

- גרעין פולינומיאלי

$$K(x, x') = (x^T x' + r)^d$$

הגרעין הכי פשוט שמביא לנו גבול ההחלטה לא לינארי.

- גרעין גאוסיאני (RBF)

$$K(x, x') = e^{-\gamma \|x - x'\|^2}$$

ממפה את הדאטה "למרחב ממימד אינסופי" ומביא גבול ההחלטה לא לינארי.

- גרעין סיגמואידי

$$K(x, x') = \tanh(\gamma x^T x' + r)$$

אם אנחנו רוצים לבנות $kernel$ משלנו, זה מאוד פשוט.

הגדרה 4.9. פונקציה $K : X \times X \rightarrow \mathbb{R}$ תיקרא **גרעין** אם היא:

1. סימטרית: $K(x, z) = K(z, x)$ לכל $x, z \in X$.
2. חיובית: במובן שלכל $x_1, \dots, x_n \in X$ קבוצה סופית מתקיים

$$\sum_{i=1}^n \sum_{j=1}^n c_i c_j K(x_i, x_j) \geq 0$$

לכל $c_i \in \mathbb{R}$

הערה 5.9. הגרעין הסיגמואידי אינו גרעין!

10 סיווג לא בינארי Multi-class Classification.

כעת, בשונה מסיווג בינארי, כאן יש לנו כמות מחלקות גדולה מ-2, כלומר $y_i \in \{c_1, c_2, c_3, \dots, c_k\}$ (זכרו כי יש לי כמות סופית של מחלקות). בסיווג בינארי הייתי צריך מודל סיווג בודד, שיחזה לי מה הסיכויים להיות במחלקה א' (ומכאן גם מה הסיכויים לא להיות במחלקה א'). כאן, נצטרך יותר ממסווג בודד, וכמות המסווגים תלוייה בטכניקת הסיווג שלנו:

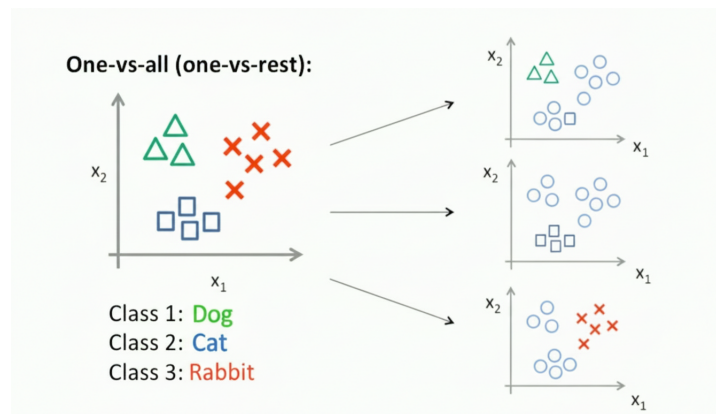
- **One vs. All** שאומר שעבור k מחלקות נצטרך לאמן k מסווגים בינאריים.
 - **One vs. One** שאומר שעבור k מחלקות נצטרך לאמן $\frac{k(k-1)}{2}$ מסווגים לא בינאריים.
- אפשר גם לסווג מודל שישיר יעשה סיווג ליותר מ-2 מחלקות בלי לעבור דרך סיווג בינארי.

10.1 סיווג One vs. All.

בשיטת אחד מול כולם, נייצר k מסווגים בינאריים עבור k מחלקות אפשריות, כך שהמסווג ה- i יסווג בין המחלקה ה- i לבין כל שאר המחלקות.

דוגמה 1.10. נניח ויש לנו תמונה ואנחנו רוצים לדעת אם זו תמונה של כלב, חתול או ארנב, אזי בשיטת OvA נייצר 3 מסווגים:

1. {כלב} מול {חתול, ארנב}.
2. {חתול} מול {כלב, ארנב}.
3. {ארנב} מול {כלב, חתול}.



דוגמה לאיך שזה נראה בפועל, יש לנו את השאתה המקורי וכל פעם אנחנו בוחרים קבוצה להיות קבוצה א' וכל שאר הדאטה להיות קבוצה ב', ומאמנים 3 מודלי סיווג בינארי על שלושת החלוקות האפשריות.

איך זה נראה בדאטה שלנו? נניח הדאטהסט שלנו הוא

חיה y	פיצ'רים X
כלב	- - - -
ארנב	- - - -
ארנב	- - - -
חתול	- - - -
כלב	- - - -
חתול	- - - -

אז נמיר את עמודת y ב-3 עמודות חדשות, כל אחת עם ערכים 0/1 שיסמנו האם הדגימה שלנו היא המחלקה הרצויה:

פיצ'רים X	כלב	חתול	ארנב
- - - -	1	0	0
- - - -	0	0	1
- - - -	0	0	1
- - - -	0	1	0
- - - -	1	0	0
- - - -	0	1	0

ואז המודל הראשון, שמסווג כלב מול כל השאר יתאמן על הדאטהסט

פיצ'רים X	כלב
- - - -	1
- - - -	0
- - - -	0
- - - -	0
- - - -	1
- - - -	0

והמודל השני, שמסווג בין חתול וכל השאר יתאמן על הדאטהסט

פיצ'רים X	חתול
- - - -	0
- - - -	0
- - - -	0
- - - -	1
- - - -	0
- - - -	1

והמודל השלישי, שמסווג בין ארנב וכל השאר יתאמן על הדאטהסט

פיצ'רים X	ארנב
- - - -	0
- - - -	1
- - - -	1
- - - -	0
- - - -	0
- - - -	0

הערה 2.10. בעצם לקחנו את עמודת y ועשינו לה *one-hot encoding*, ואימנו מודל אחד על כל עמודה חדשה שיצאה.

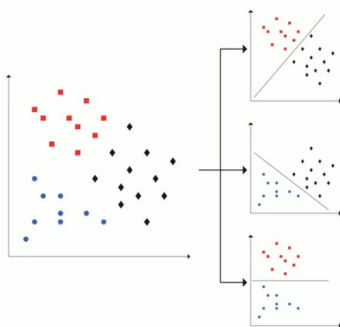
הערה 3.10. לעשות *one-hot encoding* לעמודה y עם מחלקות $c_1, \dots, c_k$ זה פשוט לקחת את כל המחלקות שבה, לכל אחת לתת עמודה חדשה משלה (k עמודות חדשות) ולכל דגימה, בעמודה ה- i לשים 1 אם בעמודה המקורית y היא הייתה c_i ו-0 אחרת.

נניח וקיבלנו על תמונה חדשה שהמסווג של כלב החזיר סיכוי 0.9 להיות כלב, המסווג של חתול החזיר סיכוי של 0.5 להיות חתול והמסווג של ארנב החזיר סיכוי של 0.4 להיות ארנב. קל, פשוט ניקח את המחלקה עם הערך הכי גבוה להיות המחלקה שאנחנו חוזים, לכן נגיד שהתמונה הזאת היא תמונה של כלב.

10.2 סיווג One vs. One.

בשיטת אחד מול אחד, ניקח כל 2 מחלקות, ונאמן מסווג בינארי רק על הדגימות של שתיהן, ככה שנתמקד בלהפריד בין שתי המחלקות האלו (בשונה מאחד מול כולם ששם התמקדנו בלהפריד מחלקה משאר המחלקות). כיוון שאנחנו לוקחים כל מחלקה מול כל מחלקה, נצטרך לאמן $\binom{k}{2} = \frac{k(k-1)}{2}$ מסווגים.

בדוגמה מעל, עם הכלב, חתול, וארנב, זה יראה כך:



כאן ספציפית יש לנו גם 3 מקרים כי $\binom{3}{2} = 3$, אבל אם היינו מוסיפים מחלקה "תרנגולת", בשיטת *OvA* היינו מאמנים עוד מודל של תרנגולת מול השאר, בעוד שפה היינו צריכים לאמן עוד 3 מודלים (תרנגולת מול כלב, תרנגולת מול חתול, ותרנגולת מול ארנב).

מבחינת הדאטה שלנו נפרק אותו שוב לטבלאות שונות:

חיה y	פיצ'רים X			
כלב	-	-	-	-
חתול	-	-	-	-
כלב	-	-	-	-
חתול	-	-	-	-

כלב מול חתול:

חיה y	פיצ'רים X			
ארנב	-	-	-	-
ארנב	-	-	-	-
חתול	-	-	-	-
חתול	-	-	-	-

חתול מול ארנב:

חיה y	פיצ'רים X			
כלב	-	-	-	-
ארנב	-	-	-	-
ארנב	-	-	-	-
כלב	-	-	-	-

כלב מול ארנב:

עכשיו אנחנו מקבלים וקטור לא של סיכויים מכל מסווג, אלא של תחזיות מכל מסווג, לדוגמה כאן בהיתן דוגמה חדשה נקבל וקטור סיכויים

$$\begin{pmatrix} \text{dog} \\ \text{cat} \\ \text{dog} \end{pmatrix}$$

לכן נסווג סה"כ את התמונה להיות המחלקה עם רוב הסיכויים, כלומר שוב פעם נסווג כאן את התמונה להיות תמונה של כלב.

10.3 סיווג Softmax regression.

נניח ואני לא רוצה לחזור שוב למסווגים בינאריים. פה אנחנו רוצים לאמן מודל אחד שיעבור על כל הדאטה שלנו, ובסוף יוציא לנו וקטור מאורך k שיגיד מה הסיכוי להיות בכל מחלקה.

כמו ברגרסיה לוגיסטית, גם כאן יש לי דרישות מהמודל: עבור מחלקות $\{c_1, \dots, c_k\}$ נרצה שהמודל יוציא פלט $(p_1, \dots, p_k)^T$ שיקיים:

$$1. \forall i \in \{1, \dots, k\} : p_i \in [0, 1]$$

$$2. \sum_{i=1}^k p_i = 1$$

פונקציה שאנחנו מכירים שנותנת לנו את זה היא פונקציית סופטמקס:

$$s(z_1, \dots, z_k) = \left(\frac{e^{z_1}}{\sum_{i=1}^k e^{z_i}}, \dots, \frac{e^{z_k}}{\sum_{i=1}^k e^{z_i}} \right)$$

מכאן שהאיבר ה- i ב- $s(\vec{v})$ הוא $\mathbb{P}(y = i|x, \theta)$. כמו ברגרסיה לוגיסטית, נגדיר את המודל שלי להיות

$$p_\theta(x) = s(w^T x + b)$$

10.3.1 פונקציית loss.

נזכור כי כל מודל הסתברותי שואף למזער את $\mathcal{L}(\theta) = - \sum_{i=1}^N \log \mathbb{P}(y_i|x_i; \theta)$ נסמן $\mathbb{P}(y_i|x_i; \theta) = p_{i,y_i}$ כאשר $p_{i,k} = [p_\theta(x_i)]_k$ זה האיבר במקום ה- k בוקטור הסופטמקס של הדגימה ה- i . נסמן

$$y_{i,k} = \begin{cases} 1 & y_i = k \\ 0 & else \end{cases}$$

לכן

$$\mathbb{P}(y_i|x_i, \theta) = \prod_{j=1}^k p_{i,j}^{y_{i,j}}$$

נציב ב- $loss$ ונקבל

$$\mathcal{L}(\theta) = - \sum_{i=1}^N \sum_{j=1}^k y_{i,j} \log p_{i,j}$$

הגדרה 4.10. אם נמצע את $\mathcal{L}(\theta)$ נקבל את **הפסד האנטרופיה הצולבת** (באנגלית - *Cross Entropy Loss*) להיות

$$CE(\theta) = - \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^k y_{i,j} \log p_{i,j}$$

10.4 איך מאמנים?

נתחיל דווקא מ-*Softmax regression*: כמובן שנרצה להשתמש ב-*GD*. בשביל זה נצטרך לחשב את הנגזרות לפי w ולפי b : נסמן, עבור הדגימה ה- i והמחלקה ה- j :

$$\begin{aligned} \bullet \quad z_{i,j} &= w_j^T x_i + b_j \\ \bullet \quad p_{i,m} &= \frac{e^{z_{i,m}}}{\sum_{r=1}^k e^{z_{i,r}}} \end{aligned}$$

$$\bullet \quad \ell_i(\theta) = - \sum_{m=1}^k y_{i,m} \log p_{i,m}$$

לכן ה- $loss$ יהיה

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_i(\theta)$$

לכל i קבוע:

$$\ell_i(\theta) = - \sum_{m=1}^k y_{i,m} \log p_{i,m}$$

$$\frac{\partial}{\partial p_{i,j}} \ell_i(\theta) = \frac{\partial}{\partial p_{i,j}} \left(- \sum_{m=1}^k y_{i,m} \log p_{i,m} \right) = - \sum_{m=1}^k y_{i,m} \frac{\partial}{\partial p_{i,j}} \log p_{i,m}$$

אבל אם $m \neq j$ נקבל כי $\log p_{i,m}$ לא תלוי ב- $p_{i,j}$ ולכן $\frac{\partial}{\partial p_{i,j}} \log p_{i,m} = 0$, כלומר נישאר רק עם $m = j$ ומכאן

$$- \sum_{m=1}^k y_{i,m} \frac{\partial}{\partial p_{i,j}} \log p_{i,m} = -y_{i,j} \frac{\partial}{\partial p_{i,j}} \log p_{i,j} = -\frac{y_{i,j}}{p_{i,j}}$$

כעת:

$$p_{i,m} = \frac{e^{z_{i,m}}}{\sum_{r=1}^k e^{z_{i,r}}}$$

נסמן $S_i = \sum_{r=1}^k e^{z_{i,r}}$. לכן $p_{i,m} = \frac{e^{z_{i,m}}}{S_i}$. נגזור:

$$\frac{\partial p_{i,m}}{\partial z_{i,j}} = \frac{\partial}{\partial z_{i,j}} \left(\frac{e^{z_{i,m}}}{S_i} \right) = \frac{S_i \frac{\partial}{\partial z_{i,j}} e^{z_{i,m}} - e^{z_{i,m}} \frac{\partial}{\partial z_{i,j}} S_i}{S_i^2}$$

אם $m = j$:

$$\begin{aligned} \frac{\partial}{\partial z_{i,j}} e^{z_{i,m}} &= \frac{\partial}{\partial z_{i,j}} e^{z_{i,j}} = e^{z_{i,j}} \\ \frac{\partial}{\partial z_{i,j}} S_i &= \frac{\partial}{\partial z_{i,j}} \sum_{r=1}^k e^{z_{i,r}} = e^{z_{i,j}} \end{aligned}$$

לכן

$$\frac{\partial p_{i,m}}{\partial z_{i,j}} = \frac{e^{z_{i,j}} S_i - e^{z_{i,j}} e^{z_{i,j}}}{S_i^2} = \frac{e^{z_{i,j}}}{S_i} \left(1 - \frac{e^{z_{i,j}}}{S_i} \right) = p_{i,j} (1 - p_{i,j})$$

ואם $m \neq j$:

$$\frac{\partial}{\partial z_{i,j}} e^{z_{i,m}} = \frac{\partial}{\partial z_{i,j}} S_i = e^{z_{i,j}}$$

לכן

$$\frac{\partial p_{i,m}}{\partial z_{i,j}} = \frac{0 \cdot S_i - e^{z_{i,m}} e^{z_{i,j}}}{S_i^2} = -\frac{e^{z_{i,m}}}{S_i} \frac{e^{z_{i,j}}}{S_i} = -p_{i,m} p_{i,j}$$

כלומר

$$\frac{\partial p_{i,m}}{\partial z_{i,j}} = p_{i,j} (\delta_{j,m} - p_{i,m})$$

סה"כ

$$\begin{aligned} \frac{\partial \ell_i(\theta)}{\partial z_{i,j}} &= \sum_{m=1}^k \frac{\partial \ell_i(\theta)}{\partial p_{i,m}} \frac{\partial p_{i,m}}{\partial z_{i,j}} \\ &= \sum_{m=1}^k -\frac{y_{i,m}}{p_{i,m}} p_{i,j} (\delta_{j,m} - p_{i,m}) \\ &= -p_{i,j} \sum_{m=1}^k \frac{y_{i,m}}{p_{i,m}} (\delta_{j,m} - p_{i,m}) \\ &= -p_{i,j} \sum_{m=1}^k \left(\frac{y_{i,m}}{p_{i,m}} \delta_{j,m} - y_{i,m} \right) \\ &= -p_{i,j} \frac{y_{i,j}}{p_{i,j}} + p_{i,j} \underbrace{\sum_{m=1}^k y_{i,m}}_{=1} \\ &= -y_{i,j} + p_{i,j} \end{aligned}$$

$$\frac{\partial z_{i,j}}{w_j} = x_i, \frac{\partial z_{i,j}}{b_j} = 1$$

כלומר

$$\begin{aligned} \frac{\partial \ell_i(\theta)}{\partial w_j} &= \frac{\partial \ell_i(\theta)}{\partial z_{i,j}} \frac{\partial z_{i,j}}{w_j} = (p_{i,j} - y_{i,j}) x_i \\ \frac{\partial \ell_i(\theta)}{\partial b_j} &= \frac{\partial \ell_i(\theta)}{\partial z_{i,j}} \frac{\partial z_{i,j}}{b_j} = p_{i,j} - y_{i,j} \end{aligned}$$

ומכאן

$$\begin{aligned} \frac{\partial L(\theta)}{\partial w_j} &= \frac{1}{N} \sum_{i=1}^N \frac{\partial \ell_i(\theta)}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (p_{i,j} - y_{i,j}) x_i \\ \frac{\partial L(\theta)}{\partial b_j} &= \frac{1}{N} \sum_{i=1}^N \frac{\partial \ell_i(\theta)}{\partial b_j} = \frac{1}{N} \sum_{i=1}^N (p_{i,j} - y_{i,j}) \end{aligned}$$

וכלל העדכון נשאר כרגיל.

עבור שיטת *One vs. All*: אם לדוגמה יש לי 3 מחלקות: כלב חתול וארנב, ואנני מכניס למודל שלי תמונה של כלב, הייתי רוצה שהמודל יוציא וקטור $\begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$ כלומר שבהסתברות 1 הוא באמת יחזה שזה כלב, ובהסתברות 0 הוא יחזה שזה חתול ובהסתברות 0 הוא יחזה שזה ארנב. אותו

היגיון כשאני מכניס תמונה של חתול, הייתי רוצה מודל שמוציא וקטור הסתברויות $\begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$, וכשאני מכניס תמונה של ארנב הייתי רוצה שהמודל

יוציא וקטור $\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$. נחזור לטבלת ה-*one-hot* שיצרנו

פיצ'רים X	כלב	חתול	ארנב
- - - -	1	0	0
- - - -	0	0	1
- - - -	0	0	1
- - - -	0	1	0
- - - -	1	0	0
- - - -	0	1	0

ונראה שבמילים אחרות אני רוצה שהמודל יפלוט לי את וקטור ה-*one-hot* של אותה השורה. מכאן שהמסווג של כלב יתאמן על $\{(x_i, y_i == \text{dog})\}_{i=1}^N$, המודל של חתול יתאמן על $\{(x_i, y_i == \text{cat})\}_{i=1}^N$ והמודל של ארנב יתאמן על $\{(x_i, y_i == \text{rabbit})\}_{i=1}^N$.

באופן יותר פורמלי, בהינתן k מחלקות $\{c_1, \dots, c_k\}$ ו- N דגימות $\{(x_i, y_i)\}_{i=1}^N$, המסווג של המחלקה ה- j מול השאר יתאמן על

$$\{(x_i, \delta_{y_i,j})\}_{i=1}^N$$

כאשר

$$\delta_{y_i,j} = \begin{cases} 1 & y_i = j \\ 0 & y_i \neq j \end{cases}$$

וכל אחד מהמודלים הללו יתאמן כרגיל.

עבור *One vs. One*:

כאן אנחנו מאמנים $\frac{k(k-1)}{2}$ מודלים, אבל לכל אחד אנחנו לא מביאים את כל הדאטה, אלא רק דגימות (x, y) עם $y \in \{i, j\}$ (כשעושים מחלקה i מול מחלקה j). לכן כעת בעמודות y החדשה שלי יהיו רק שני ערכים, i ו- j , כלומר אפשר להתייחס לזה כתור אימון מודל סיווג בינארי לכל מודל בנפרד. פורמלית, אם אנחנו עושים מחלקה i מול מחלקה j , אז הדאטהסט שלנו יהיה

$$\{(x_k, y_k) | y_k \in \{i, j\}\}_{k=1}^N$$

כאשר נסמן מחדש

$$y'_k = \begin{cases} 1 & y_k = i \\ 0 & y_k = j \end{cases}$$

ואז המודל יתאמן על $\{(x_k, y'_k)\}$ כמו מודל סיווג בינארי רגיל.

11 הערכת המודל המסווג.

נניח ויש לנו דאטה עד הרבה אנשים שבאו להיבדק למחלת לב, ואנחנו רוצים לנבא בהינתן מטופל חדש האם הוא יחלה במחלת לב או לא. כיוון שאנחנו לא מדעני נתונים עצלנים, אנחנו לא רוצים סתם לאמן מודל עלוב על הדאטה שלנו ולסיים בזה, אנחנו רוצים לבנות את המודל הכי טוב על הדאטה שלנו.

נשאלת השאלה, מה הכוונה בהכי טוב? איך אנחנו יכולים לכמת במספר כמה המודל שלנו טוב, בדרך שתאפשר לנו להשוות בין כמה מודלים?

11.1 מטריצת בלבול - Confusion Martix.

נחלק את הדאטה שלנו ל-*train* ו-*test* ונאמן מודל סיווג בינארי על ה-*train*. עכשיו אני רוצה לראות כמה צדקתי על ה-*test*. דרך אחת לעשות את זה, היא בעזרת שימוש במטריצת בלבול (*Confusion Martix* באנגלית). היא נראית ככה:

		תיוג אמיתי	
		יש מחלת לב	אין מחלת לב
תיוג מהמכונה	יש מחלת לב		
	אין מחלת לב		

השורות במטריצת הבלבול הן מה שהמכונה חזתה, והעמודות במטריצת הבלבול תהיינה התיוג האמיתי של המטופל.

כיוון שיש רק 2 תיוגים לבחור מהם, התא השמאלי העליון יכול דגימות True Positive - אלו מטופלים שבאמת היה להם מחלת לב והמכונה זיהתה שיש להם מחלת לב.

דגימות בתא הימני התחתון יהיו True Negative - אלו מטופלים שבאמת אין להם מחלת לב והמכונה אמרה שאין להם מחלת לב.

דגימות בתא השמאלי התחתון יהיו False Negative - אלו מטופלים שיש להם מחלת לב אבל המכונה קבעה שאין להם מחלת לב.

ולבסוף, דגימות בתא הימני העליון יהיו False Positive - אלו מטופלים שאין להם מחלת לב אבל המכונה קבעה שיש להם מחלת לב.

לסיכום, מטריצת בלבול תיראה כך

	Actual Positive	Actual Negative
Predicted Positive	True Positive (TP)	False Positive (FP)
Predicted Negative	False Negative (FN)	True Negative (TN)

הבחנות:

- הערכים על האלכסון אומרים לנו כמה פעמים המודל סיווג נכון.
- הערכים שלא על האלכסון אומרים לנו כמה פעמים המודל לא סיווג נכון.
- סך כל הערכים במטריצת הבלבול ($TP + FP + TN + FN$) זה כך כל הדגימות שלי, כלומר כל דגימה הולכת לתא אחד ויחיד במטריצה.

דוגמה 1.11. אחרי שסיווגנו לחולה/ לא חולה לב, קיבלנו את התוצאות הבאות

		תיוג אמיתי	
		יש מחלת לב	אין מחלת לב
תיוג מהמכונה	יש מחלת לב	142	22
	אין מחלת לב	29	110

לכן אנחנו יודעים להגיד על המסווג שלנו שהוא סיווג נכון 142 מטופלים שהיה להם מחלת לב True Positive (וסיווג שהיה להם מחלת לב), ושהוא סיווג נכון 110 מטופלים שלא היה להם מחלת לב True Negative (והוא סיווג שלא היה להם מחלת לב).

לעומת זאת, האלגוריתם טעה בסיווג של 29 מטופלים בכך שאמר שאין להם מחלת לב כשבועצם יש להם מחלת לב False Negative, ובעוד 22 מטופלים שאמר שיש להם מחלת לב כשבועצם לא היה להם False Positive.

עכשיו אנחנו יכולים לאמן כמה מודלים, ולהשוות ביניהם בעזרת מטריצות הבלבול שלהם:

נאמן את שני המודלים על אותו *train* ונקבל את התוצאות הבאות:

		תיוג אמיתי	
		יש מחלת לב	אין מחלת לב
תיוג מהמכונה	יש מחלת לב	142	22
	אין מחלת לב	29	110

מודל רגרסיה לוגיסטית:

לעומת

		תיוג אמיתי	
		יש מחלת לב	אין מחלת לב
תיוג מהמכונה	יש מחלת לב	107	53
	אין מחלת לב	64	79

מודל KNN:

המודל KNN גרוע יותר בלסווג חולים עם מחלת לב, אנחנו שמים לב לזה כי כמות ה-True Positive שלנו ירדה ב-35 חולים, ולעומת זאת הם עברו ללהיות מסווגים כתור לא בעלי מחלת לב, כלומר כמות ה-False Positive עלתה ב-35 מטופלים.

בנוסף, מודל ה-KNN גרוע גם בלסווג חולים בלי מחלת לב, כי הוא סיווג 31 מטופלים יותר כתור False Negative מאשר מודל הרגרסיה הלוגיסטית שלנו.

לכן, אם נצטרך לבחור בין מודל ה- KNN למודל הרגרסיה הלוגיסטית, נבחר במודל הרגרסיה הלוגיסטית.

עכשיו נניח שיש לנו דאטה חדש, תמונות של בעלי חיים ואני רוצה לדעת האם התמונה מציגה חתול, כלב, או ארנב. מטריצת הבלבול שלי תיראה ככה:

		תיוג אמיתי		
		כלב	חתול	ארנב
תיוג המכונה	כלב	112	6	0
	חתול	12	102	4
	ארנב	8	60	98

בדיוק כמו בסיווג בינארי, הערכים על האלכסון אומרים לי כמה דגימות נכונות סה"כ סיווגתי, ושאר הערכים (אלו שלא על האלכסון) אומרים לי כמה דגימות סיווגתי לא נכון.

11.2 ROC, AUC ושאר חברים.

עכשיו נניח יש לי עבור מודל מסויים את ה- TP, TN, FP, FN שלו, אני יכול לשאול שאלות יותר מתקדמות על המודל שלי, כגון:

1. $Precision$: מבין כל התחזיות שחזו חיוביות לפי המודל שלי, כמה מהן באמת חיוביות? אז ניקח כל מי שסומן True Positive ונחלק בכל מי שסומן בכללי Positive:

$$Precision = \frac{TP}{TP + FP} = \mathbb{P}(y = 1 | \hat{y} = 1)$$

2. $Recall$: מבין כל התחזיות שבאמת חיוביות, כמה מהן חזו חיוביות לפי המודל שלי? אז ניקח כל מי שסומן True Positive ונחלק בכל מי שבאמת Positive:

$$Recall = \frac{TP}{TP + FN} = \mathbb{P}(\hat{y} = 1 | y = 1)$$

לעיתים גם יקרא $TPR - True Positive Rate$.

3. FPR : מבין כל הדגימות שבאמת שליליות, כמה מהן המודל שלי סיווג לא נכון? אז ניקח כל מי שסומן False Negative ונחלק בכל מי שבאמת Negative:

$$FPR = \frac{FP}{FP + TN} = \mathbb{P}(\hat{y} = 1 | y = 0)$$

$FPR - False Positive Rate$.

4. $Specificity$: מבין כל הדגימות שבאמת שליליות, כמה מהן המודל שלי סיווג נכון?

$$Specificity = \frac{TN}{FP + TN} = 1 - FPR = \mathbb{P}(\hat{y} = 0 | y = 0)$$

לעיתים גם יקרא $TNR - True Negative Rate$.

5. $F_1 - Score$: ממוצע הרמוני של $Precision$ ו- $Recall$:

$$F_1 = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}}$$

6. $Accuracy$: החלק היחסי של התחזיות הנכונות מתוך כלל הדגימות.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \mathbb{P}(y = \hat{y})$$

7. $ROC curve$: בסיווג בינארי, ראינו שכדי לסווג דגימה כחיובית היינו צריכים לקבוע $threshold$ מסויים שלפיו מסווגים את הדגימה. ב- SVM זה היה 0, וברגרסיה לוגיסטית זה היה 0.5. אבל, זה רק ערך בודד מפני ∞ ערכי $threshold$ אפשריים. נרצה לבחון את ביצועי המודל לאורך מספר $thresholds$ כי אנחנו נרצה את ה- $threshold$ הכי טוב עבור הבעיה שלנו (אבל אין באמת ערך שהוא טוב תמיד, אלא תמיד נבחר פר בעיה). נגדיר את ה- $ROC curve$ להיות עקומה המציגת על ציר ה- y את ה- TPR , ועל ציר ה- x את ה- FPR , לכל ערכי ה- $threshold$ האפשריים.

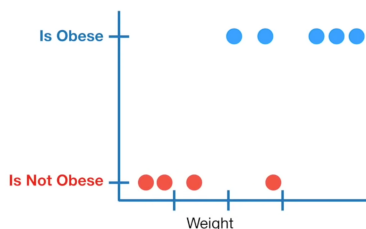
8. AUC : השטח מתחת לעקומת ה- ROC .

$$AUC = \mathbb{P}(\text{model ranks a random positive higher than a random negative})$$

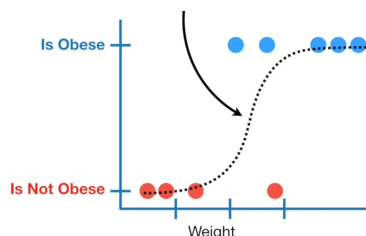
אומר לנו כמה המודל יכול להבחין בין מחלקות. AUC גבוה יותר אומר שהמודל מסווג יותר טוב (כלומר בכל מודל אנחנו מעוניינים שה- AUC שלו יהיה כמה שיותר קרוב ל-1).

שאלה 2.11. בהינתן מודל סיווג בינארי עם $AUC < \frac{1}{2}$, מדוע אופרים שמודל יותר טוב הוא אחד שמחשש ברנדומליות 0 או 1 ($\mathbb{P}(0) = \mathbb{P}(1) = 0.5$)?

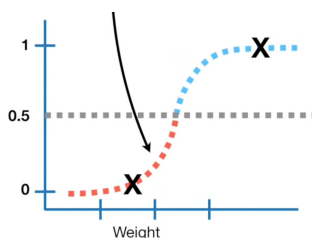
דוגמה 3.11. אינטואיציה מאחורי AUC ו- ROC curve: נניח כי שוב פעם יצאתי למדוד משקל של עכברים ואני רוצה לסווג לפי זה האם הם שמנים או לא שמנים, ויצאו לי הנקודות האלו:



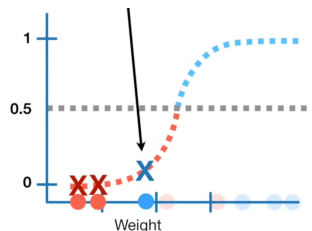
יש לנו עכבר שלא שמן, למרות שיש לו משקל גבוה (כנראה שזה עכבר ממש שרירי), ויש לנו עכבר רזה שכן שמן. נתאים מודל רגרסיה לוגיסטית לדאטה שלנו ונקבל את הדבר הבא:



עכשי וציר ה- y הפך להיות ההסתברות של דגימה חדשה להיות שמנה. ככל שאני דוגם עכבר כבד יותר, סיכוי גבוה יותר שנסווג אותו להיות שמן, וככל שאני דוגם עכבר קל יותר, ככה סיכוי נמוך יותר שאני אסווג אותו כשמן. אמרנו כבר שבשביל שאני אסווג לשמן/ לא שמן צריך לקבוע $threshold = 0.5$ לדוגמה נקבע $threshold = 0.5$ ונקבל



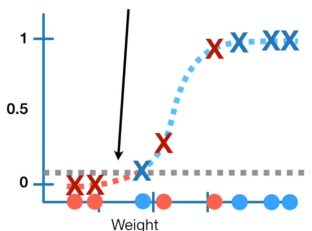
כלומר כל מי שמקבל סיכוי להיות שמן יותר מ-0.5 נסווג להיות שמן, ומי שמקבל סיכוי להיות שמן $0.5 \leq$ נסווג לא להיות שמן. כדי להעריך את יעילות הרגרסיה הלוגיסטית ביחד עם $threshold = 0.5$, נבחן את המודל על ה- $test$ set:



שני העכברים השמאליים הם לא שמנים, ואכן אנחנו מסווגים אותם להיות לא שמנים, אבל הנקודה הכחולה לידם מסווגת להיות לא שמנה למרות שזה כן עכבר שמן. העכבר שאחריו מסווג כמו שצריך, אך העכבר הבא לא מסווג נכון. שלושת העכברים האחרונים סווגו נכון. לכן אנחנו יכולים לבנות מטריצת בלבול כדי לסכם את התוצאות שלנו:

תיוג אמיתי	תיוג אמיתי	
	שמן	לא שמן
תיוג מהמכונה	שמן	3
	לא שמן	1

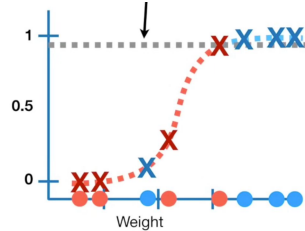
איך התוצאות ישתנו אם נשנה את ה- $threshold$ שלנו? לדוגמה, אם ממש חשוב לנו לסווג נכון את כל העכברים השמנים כשמנים, נקבע את ה- $threshold$ להיות 0.1 ונקבל



ונקבל סיווג נכון לכל ארבעת העכברים השמנים, כלומר ה- TP שלנו יעלה. כמובן שזה יגרום ל- TN שלנו לרדת, אבל כאן יצא שגם ה- FP עלה, כי לקחנו שני עכברים לא שמנים וסיווגנו אותם להיות שמנים. נקבל את מטריצת הבלבול הבאה:

		תיוג אמיתי	
		שמן	לא שמן
תיוג מהמכונה	שמן	4	2
	לא שמן	0	2

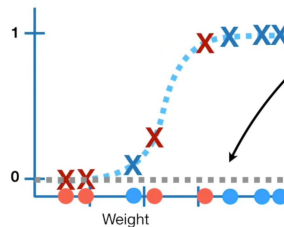
אם מצד שני היינו קובעים את ה- $threshold$ שלנו להיות 0.9 היינו מקבלים



עדיין ה- TP שלנו לא ישתנה מהאחד שהיה לנו עבור $threshold = 0.5$, אבל ה- FN שלנו עלה כי סיווגנו את כל העכברים הלא זמנים כלא שמנים, כמו שצריך. מכאן גם שירד ה- FP .

אלו רק 3 ערכי $threshold$ בין 0 ל-1, יש ∞ כאלו. איך אנחנו קובעים איזה הוא הכי טוב? תחילה, נשים לב כי צריך רק לבדוק את מטריצות הבלבול רק על $threshold$ שמנים את מטריצת הבלבול (נגיד לא נצטרך לבדוק כאן בין 0.6 ל-0.7 כי זה לא ישנה את הסיווגים שלנו. אם ניצור מטריצת בלבול לכל ערך כזה שמשה, נקבל מספר מבלבל של מטריצות בלבול (●). לכן נשרטט את ה- ROC curve: על ציר ה- y נשרטט את ה- TPR כתלות ב- $threshold$, ובציר ה- x נשרטט את ה- FPR כתלות ב- $threshold$.
נשרטט:

נתחיל מ- $threshold$ שמסווג את כל העכברים כשמנים:



ונקבל מטריצת בלבול

		תיוג אמיתי	
		שמן	לא שמן
תיוג מהמכונה	שמן	4	4
	לא שמן	0	0

נחשב FPR, TPR :

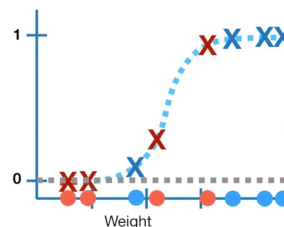
$$TPR = \frac{TP}{TP + FN} = \frac{4}{4 + 0} = 1$$

$$FPR = \frac{FP}{TN + FP} = \frac{4}{4 + 0} = 1$$

כלומר עבור $threshold$ נמוך קיבלנו שאנחנו מסווגים נכון את כל העכברים השמנים, אך טועים לגמרי בכל העכברים הלא שמנים בכך שאנחנו מסווגים אותם כתור עכברים שמנים.

נשרטט נקודה ב- $(1, 1)$. נקודה ב- $(1, 1)$ אומרת שלמרות שסיווגנו נכון את כל הדגימות של העכברים השמנים, סיווגנו לא נכון את כל הדגימות של העכברים שלא שמנים.

נגדיל את ה- $threshold$ עד שיהיה לנו רק עכבר אחד שנסווג כלא שמן:



נקבל מטריצת בלבול

		תיוג אמיתי	
		שמן	לא שמן
תיוג מהמכונה	שמן	4	3
	לא שמן	0	1

$$TPR = \frac{TP}{TP + FN} = \frac{4}{4 + 0} = 1$$

$$FPR = \frac{FP}{TN + FP} = \frac{3}{3 + 1} = 0.75$$

ונשרטט נקודה ב-(0.75, 1). זה אומר שעדיין אנחנו מסווגים נכון 100% מהדגימות של העכברים השמנים, אבל עכשיו רק 75% מהעכברים הלא שמנים מסווגים כעכברים שלנים, כלומר זהו $threshold$ טוב יותר (אובייקטיבית). נגדיל את ה- $threshold$ ככה שהפעם שני עכברים יסווגו כתור לא שמנים ונקבל מטריצת בלבול

		תיוג אמיתי	
		שמן	לא שמן
תיוג מהמכונה	שמן	4	2
	לא שמן	0	2

כלומר נקבל

$$TPR = \frac{TP}{TP + FN} = \frac{4}{4 + 0} = 1$$

$$FPR = \frac{FP}{TN + FP} = \frac{2}{2 + 2} = 0.5$$

ומכאן נשרטט (0.5, 1) כלומר שוב קיבלנו $threshold$ טוב יותר. נגדיר את ה- $threshold$, הפעם קיבלתי עכבר שמן שאסווג אותו כתור עכבר לא שמן, כלומר נקבל

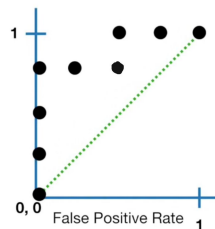
		תיוג אמיתי	
		שמן	לא שמן
תיוג מהמכונה	שמן	3	2
	לא שמן	1	2

ומכאן

$$TPR = \frac{TP}{TP + FN} = \frac{3}{3 + 1} = 0.75$$

$$FPR = \frac{FP}{TN + FP} = \frac{2}{2 + 2} = 0.5$$

כלומר נשרטט נקודה ב-(0.5, 0.75). נמשיך ככה ונקבל את הגרף הזה:



אם נחבר אותו בקו נקבל את עקומת ה- ROC לתרגיל שלנו, והשטח מתחת לגרף (ה- AUC) יהיה 0.875. אם כעת יבוא דוד ויגיד לי שהוא הריץ $Random\ forest$ על הדאטה שלי ויצא לו $AUC = 0.7$, אני אדע שהמודל שלי הוא הטוב מבין השניים.

11.3 הערכת מודל מסווג לא בינארי.

קיימת הקבלה של זה לסיווג רב מחלקתי, למרות שזה לא הכי יפה: נזכור כי עבור סיווג של תמונות לכלב, חתול או ארנב המודל שלי קיבל את מטריצת הבלבול

		תיוג אמיתי		
		כלב	חתול	ארנב
תיוג המכונה	כלב	112	6	0
	חתול	12	102	4
	ארנב	8	60	98

אז אני באמת יכול לחשב $accuracy$, TPR , FPR כאן, פשוט שאני אצטרך לעשות את זה לכל מחלקה בנפרד. אם לדוגמה אני ארצה לחשב FPR לכלב, נקבל שאני צריך לעבוד עם התוצאות

תיוג אמיתי		
	כלב	חתול+ארנב
תיוג המכונה	כלב	$TP = 112$
	חתול+ארנב	$FN = 12$
		$FP = 6$
		$TN = 264$

לכן

$$FPR_{dog} = \frac{6}{6 + 264} = 0.022$$

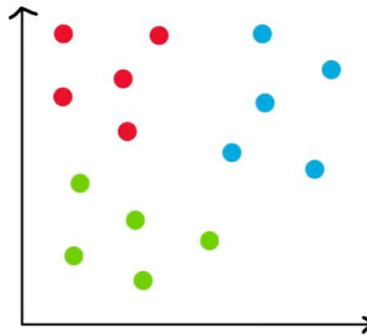
12 אלגוריתם KNN.

עד כה, הגדרנו מודל באותה צורה:

מגדירים בעיה $\leftarrow$ בונים מודל $\leftarrow$ מגדירים הפונקציית $loss$ $\leftarrow$ מאפסמים את הפרמטרים של המודל.

מה אם אנחנו לא נגדיר מודל?

שאלה 1.12. בהינתן הנקודות הבאות:



איך אנחנו יכולים לסווג נקודה חדשה?

תשובה: פשוט נסווג אותה לפי התוויות של השכנים שלה!

הרעיון הוא מאוד פשוט: נקבע מראש קבוע k שלפיו נסווג את הנקודה.

אלגוריתם 2.12. בהינתן נקודה חדשה:

1. חשבו את המרחק שלה לכל נקודה אחרת.
 2. מצאו את k הנקודות הכי קרובות אל הנקודה שלנו.
 3. סווגו את הנקודה החדשה אל המחלקה שבה נמצאים הכי הרבה שכנים שלה
- וזהו! בנינו את אלגוריתם KNN !

הערה 3.12. בשלב (2), אנחנו צריכים לחשב את המרחק בין 2 דגימות, זה יכול להיות מאוד מסובך אם:

1. לא כל הדגימות שלי מקבלות אותו משקל.
 2. היחידות מידה של הדגימות שונות.
 3. יש לנו דגימות שלא תורמות לסיווג.
- לכן אנחנו צריכים להגדיר **מטריקה** על הדגימות שלנו.

שאלה 4.12. האם כל מטריקה תעבוד?

תשובה: לא! הפטריקה שאנחנו משתמשים בה צריכה לבטא קרבה בין הדאטה שלנו. האלגוריתם מניח שנקודות קרובות אחת לשנייה דומות מבחינת סיווג, לכן אנחנו צריכים להביא לו מטריקה שמשקפת את זה.

שאלה 5.12. האם אנחנו יכולים להשתמש ב- KNN לעוד משימה?

תשובה: כן! אם אני מעוניין במשימת רגרסיה, אני יכול פשוט לחזות את y שלי להיות ממוצע ערכי ה- y של k הדגימות הכי קרובות שלי ב- x .

אלגוריתם 6.12. בהינתן נקודה חדשה:

1. חשבו את המרחק שלה לכל נקודה אחרת.
2. מצאו את k הנקודות הכי קרובות אל הנקודה שלנו $\{y_{i_j}\}_{j=1}^k$.

$$3. \text{ החזירו } \hat{y} = \frac{1}{k} \sum_{j=1}^k y_{i_j}$$

יתרונות וחסרונות של אלגוריתם KNN :

יתרונות:

1. פשוט למימוש - רק צריך מטריקה מתאימה ולבחור את k (לא צריך $loss$, ולא GD).

2. לא צריך לאמן את המודל.

3. יכול לעבוד על דאטה לינארי, לא לינארי, וככל שיש יותר דאטה הסלט יותר מדויק.

4. עובד לסיווג ורגרסיה.

חסרונות:

1. צריך לעבור על כל הדאטה - $O(Nd)$ להחזרת פלט (d = כמות הפיצ'רים, N = כמות הדגימות שלי).
 2. יקר בזיכרון - צריך לשמור את כל הנקודות, כשבעבר היה אפשר לשמור כמות נמוכה יחסית של פרמטרים.
 3. בערכי k קטנים, *outliers* יכולים להשתלט על הדגימה, ובערכי k גדולים, קבוצות קטנות מדי יכולות להיעלם.
 4. צריך שהדאטה יהיה מאוזן (אני לא יכול לתת אותו משקל לגובה 1.8 מטר ול-1,000,000 ש"ח), אבל אין איך לעשות רגולריזציה.
 5. במימדים גבוהים מאוד (d ענקי), מרחק בין 2 נקודות מאבד משמעות.
 6. רגיש לפיצ'רים שיכולים להיות לא שימושיים - פיצ'ר אחד יכול להרוס את כל המרחק.
- איך אפשר לתקן את זה: המרחק (האוקלידי) בין 2 נקודות הוא

$$d(x, z) = \|x - z\|_2 = \sqrt{(x - z)^T (x - z)}$$

ואם אנחנו רוצים למשקל את המרחק הזה, נוכל להגדיר

$$d_W(x, z) = \sqrt{(x - z)^T W (x - z)}$$

על מנת ללמוד קשרים בין $x_i - z_i$ ובין $x_j - z_j$. כדי להבטיח שעדיין יש לנו מטריקה נחייב את W להיות חיובית על ידי $W = L^T L$ כלומר לא נלמד d_W אלא נלמד

$$d_L(x, z) = \|L(x - z)\|_2$$

הבאסה פה זה שהפעם כן צריך לאמן את W , ניתן לקוראים בבית כתור תרגיל לחשוב על פונקציית *loss* מתאימה.

שאלה 7.12. האם אנחנו יכולים להשתמש ב- KNN לעוד משימה חוץ מחיזוי y ?

תשובה: כן! אנחנו יכולים להשתמש באלגוריתם כדי להשלים דאטה חסר:

נניח שיש לי דאטה $y_i, x_i = (x_i^{(1)}, \dots, x_i^{(d)})$ אבל עבור דגימה כלשהי, אחד מהפיצ'רים לא נזגם כמו שצריך

לדוגמה:

	$x^{(1)}$	$x^{(2)}$	$x^{(3)}$	$x^{(4)}$	y
(x_1, y_1)	17	3	0	1	0
(x_2, y_2)	-9	-	0	2	1
(x_3, y_3)	4	2	1	3	1
(x_4, y_4)	7	0	1	3	0

אזי $x_2^{(2)}$ חסר. נוכל לחשב את ה- k דגימות הכי קרובות אליו (מבחינת ערכי x שלא חסרים) $\{x_{i_j}\}_{j=1}^k$ ואז להשלים

$$x_2^{(2)} = \frac{1}{k} \sum_{j=1}^k x_{i_j}^{(2)}$$

13 קביעת היפרפרמטרים ו-Cross-validation.

אני עכשיו רוצה לבחור בין כמה מודלים. נניח המשימה שלי היא סיווג בינארי (למרות שהרעיון זהה לכל משימה אחרת).

בעבר פשוט חילקנו את הדאטה ל- $train \backslash test$ ולקחנו את המודל עם ה- AUC הכי גבוה.

Data

Train Test

הבעיה היא שלחלק את הדאטה שלנו זה משהו שאנחנו לא רוצים, היינו רוצים שיהיה לנו $test$ כמה שיותר גדול כדי שתהיה לנו הערכה אמפירית כמה שיותר אמיתית לאיכות המודל (גם ככל שהדאטה יותר קטן, לחלק את הדאטה שלי עוד יותר מכניס שאלה של מה היה קורה אם הייתי דוגם אחרת ל- $test$). אני לא יכול לבדוק את הדאטה שלי על ה- $train$ כי אז זה לא ישקף לי כמה המודל מכליל, אבל אני לא יכול לבדוק את המודל שלי על כל הדאטה כי אז אני לא אאמן את המודל שלי. מה נעשה?

אנחנו יכולים לחלק את הדאטה שלנו ל-5 קבוצות הנקראות *Folds*

Data

Fold 1 Fold 2 Fold 3 Fold 4 Fold 5

ואז אם בעבר הייתי משתמש ב- $Fold$ החמישי בשביל ה- $test$, אני עכשיו יכול להשתמש בכל ה- $Folds$ שלי. איך?

פשוט נעבור על כולן, ובכל פעם נבחר אותה להיות ה- $test$ שלנו.

עכשיו אימנו 5 מודלים ויש לנו חמישה AUC , מה נעשה איתם? נחשב להם ממוצע וסטיית תקן, כאשר הממוצע יעריך לנו את ה- AUC האמיתי של המודל וסטיית התקן תאמר לנו כמה לסמוך על הממוצע.

לתהליך קוראים $Cross - validation$ או בקיצור CV ויראה ככה:

	Data				
model 1	Train	Train	Train	Train	Test
model 2	Train	Train	Train	Test	Train
model 3	Train	Train	Test	Train	Train
model 4	Train	Test	Train	Train	Train
model 5	Test	Train	Train	Train	Train

ככה יצא שאני בוחן את המודל שלי על כל הדאטה, בלי לאמן ולבחון על אותה דגימה בו זמנית.

כשאני מחלק את הדאטה שלי ל-5 קבוצות קוראים לתהליך $5 Fold Cross Validation$, ובכללי כשמחלקים ל- k קבוצות קוראים לתהליך $k - Fold Cross - Validation$.

אני עכשיו רוצה למצוא את ההיפרפרמטרים הכי טובים למודל שלי (יכול להיות d דרגת הפולינום בגרסיה פולינומיאלית או λ קבוע הרגולריזציה או k כמות השכנים שאני משתמש בהם ב- KNN) איך נעשה את זה?

הפתרון נורא פשוט: נגדיר טווח לכל היפרפרמטר ונדגום אותם משם, נעריך כמה טוב המודל שלנו בזכותם ובסוף נבחר את הצירוף הכי טוב

דוגמה 1.13. נניח אני מאמן מודל גרסיה פולינומיאלית, ששם אני יכול לקבוע את:

• d - מעלת הפולינום.

• λ_1 - עוצמת רגולריזציה l_1 .

• λ_2 - עוצמת רגולריזציה l_2 .

• k - כמה שכנים אני לוקח כדי להשלים איתם ערכים חסרים.

אזי נגדיר לכל אחד תחום שאנחנו רוצים למצוא היפרפרמטרים מתוכו:

• $d \in \{2, \dots, 9\}$

• $\lambda_1 \in \{0, 0.001, 0.01, 0.1, 1\}$

• $\lambda_2 \in \{0, 0.001, 0.01, 0.1, 1\}$

• $k \in \{5, 10\}$

נשים לב שאם הייתי מחפש בכוח על כל הצירופים האפשריים, הייתי עובר על 180 קומבינציות פה, ואם הייתי קובע $\lambda_1 \in [0, 1]$ היו לי אף אפשרויות להיפרפרמטרים (הרבה אפשרויות), לכן נדגום ברנדומליות נניח 5 פעמים ונקבל

d	λ_1	λ_2	k	MSE
6	0.1	0.1	10	2.155543
4	0.1	0.01	10	27.248841
7	1	0.001	5	5.280761
4	0.001	0.1	5	2.726469
2	0	1	10	7

אזי קל וחומר שניקח את הצירוף הראשון של $d = 6, \lambda_1 = 0.1, \lambda_2 = 0.1, 10$ ונדווח שהמודל הכי טוב שלי קיבל MSE של 2.155543.

שאלה 2.13. האם אפשר להשתמש ב- CV כדי לקבוע היפרפרמטרים?

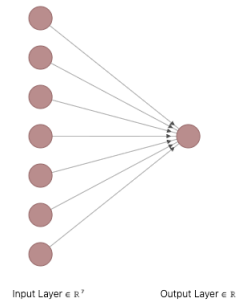
תשובה: ברור שכן, ואפילו מופלץ. נשתמש ב- $k - Fold CV$ על ה- $Train$ (אחרי שפיצלנו ל- $train \backslash test$), נדגום את ההיפרפרמטרים שלנו ונקבל הפעם ממוצע לכל דגימה, וניקח את ההיפרפרמטרים עם הממוצע הכי גבוה (בתנאי שאין לו סטיית תקן מטורפת, כמובן), ואז נעריך את המודל שלנו על ה- $test$.

חלק III

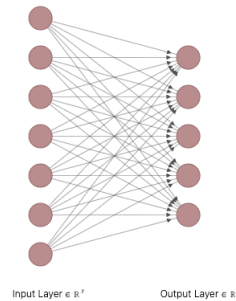
רשתות נוירונים.

14 הגדרות, אינטואיציה.

ראינו כבר שאנחנו יכולים להשתמש במבנה מודל לינארי עבור סיווג/גרסיה. עבור מודל גרסיה לדוגמה היה לנו $y_i = Wx_i + b$, שזה נראה ככה:



ועבור מודל סיווג היה לנו (במולטיקלאס):



כאשר כל קשת בין $input\ node$ ל- $output\ node$ היא משקולת w_{ij} . נרצה לפרמל קצת את המודל:
הגדרה 1.14. נוירון זה פונקציה המחשבת סכום ממושקל של כל הקלטים שלה, ואז מפעיל פונקציה f על הסכום הממושקל. במתמטית, נוירון הוא בדיוק

$$y = f(Wx + b)$$

הגדרה 2.14. f הזו נקראת **פונקציית אקטיבציה**.

הגדרה 3.14. רשת נוירונים זה פונקציה המקבלת קלט, ומעבירה אותו דרך הרבה נוירונים לצורך הוצאת פלט מסויים. גם מודל הסיווג וגם מודל הרגרסיה הם מודלים של רשתות נוירונים, ואת שניהם אפשר לסכם במשוואה

$$y = f(Wx + b)$$

על ידי בחירת f ומימדים מתאימים של W המתאימים לבעיה.

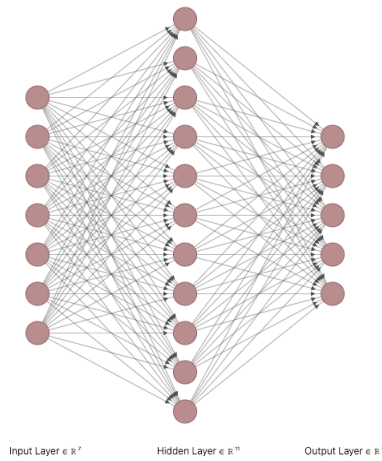
הגדרה 4.14. שכבה של נוירונים נקראת **layer**.

הגדרה 5.14. הקלט של רשת נוירונים נקרא **input layer**. הפלט של רשת נוירונים נקרא **output layer**.

הגדרה 6.14. כל שכבה של רשת נוירונים שהיא לא **input/output layer** נקראת **hidden layer**.

נשאלת השאלה, מה אם אני רוצה מודל יותר חזק?

נשאלת השאלה, למה המודל שלנו לא חזק כבר עכשיו. התשובה היא, שה-**output layer** הוא פשוט צירוף לינארי $y = Wx + b$ של כל ה-**input layer**, לדוגמה $y = 2x_1 - 0.7x_2 + 0.2x_3$, שאת זה אנחנו כבר מכירים. מה אם לדוגמה, אני ארצה לתת משקל לא רק ל- x_1 , אלא ל- $3x_1 - 2x_2$? פה במודל הפשוט שלנו אין לנו איך לעשות את זה, כיוון שיש לנו צירוף לינארי של ה-**input layer**, ואנחנו רוצים צירוף לינארי של צירוף לינארי שלהם. אז מה הפתרון? אפשר להוסיף עוד שכבה באמצע, שהיא תעזור לנו לחשב צירופים לינארים של ה-**input layer**, ואז נחשב צירופים לינארים ממנה!



עכשיו בעזרת ה-*hidden layer* אנחנו יכולים לתאר את התהליך שלנו ב-2 שלבים:

$$h = f(W^1x + b^1)$$

$$y = f(W^2h + b^2)$$

או פשוט

$$y = f(W^2f(W^1x + b^1) + b^2)$$

נשאלת השאלה, מי זו f הזו? מה אנחנו רוצים ממנה?

שאלה 7.14. האם f יכולה להיות $f(x) = x$?

תשובה: לא! אחרת המודל שלנו היה קורס להיות שכבה אחת

$$y = f(W^2f(W^1x + b^1) + b^2) = \underbrace{W^2W^1}_{=W}x + \underbrace{W^2b^1 + b^2}_{=b} = Wx + b$$

באותו אופן f לא יכולה להיות פונקציה לינארית, כיוון שאז עוד פעם המודל היה קורס להיות מודל לינארי:

$$f(x) = Ax + B \Rightarrow y = f(W^2f(W^1x + b^1) + b^2) = \dots W'x + b'$$

כלומר y כפונקציה של x היה שוב פעם בצורה מישורית, ואת זה אנחנו יודעים לעשות מעולה בלי קשר לשכבות ונוירונים. לכן אנחנו ניקח את f שלנו להיות פונקציה **לא לינארית**, על מנת שלא נוכל לצמצם את y להיות $y = \bar{W}x + \bar{b}$.

מעולה, עכשיו אנחנו יודעים לבנות מודל חזק יותר באמצעות 2 שכבות, ובאותו אופן אני יכול להגדיל את המודל שלי עד ω שכבות (ω הכוונה כל כמות סופית, אך ללא חסם עליון). ככה אני יכול ליצור ארכיטקטורות מסובכות כמו

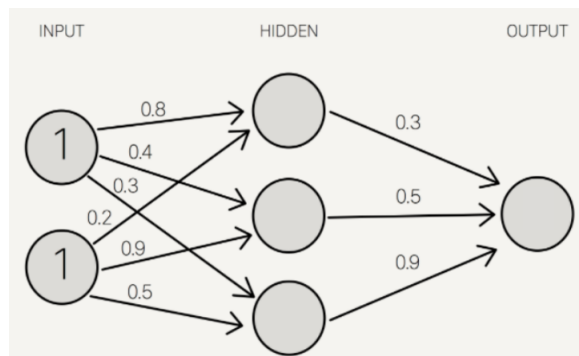
$$h^1 = f(W^1x + b^1)$$

$$h^i = f(W^i h^{i-1} + b^i)$$

$$y = f(W^k h^{k-1} + b^k)$$

עבור $i \in \{2, \dots, k-1\}$, כלומר הגדרנו פה רשת עם $k-2$ שכבות. בעצם הרעיון הוא שכל שאנחנו מגדילים את מספר השכבות, ככה אנחנו נותנים לרשת שלנו ללמוד יותר קשרים לא רק כפונקציה של הפיצ'רים, אלא גם קשרים בין הפיצ'רים (אולי לסכום $6x_1 - 0.7x_2$ יותר טוב מאשר פשוט wx_1).

דוגמה 8.14. נתונה רשת נוירונים הבאה:



עם פונקציית האקטיבציה $\sigma(x) = \frac{1}{1+e^{-x}}$. חשבו מה יהיה הפלט של הרשת.
פתרון. נחשב את הערך הראשון:

$$\begin{aligned} 1 \cdot 0.8 + 1 \cdot 0.2 &= 1 \Rightarrow \sigma(1) = 0.73 \\ 1 \cdot 0.4 + 1 \cdot 0.9 &= 1.3 \Rightarrow \sigma(1.3) = 0.78 \\ 1 \cdot 0.3 + 1 \cdot 0.5 &= 0.8 \Rightarrow \sigma(0.8) = 0.68 \end{aligned}$$

וכדי לחשב את הפלט:

$$0.73 \cdot 0.3 + 0.78 \cdot 0.5 + 0.68 \cdot 0.9 = 1.221 \Rightarrow \sigma(1.221) = 0.77$$

וזה הפלט של הרשת.

15. Back-propagation

עכשיו בעצם כשיש לנו מודל שאנחנו יודעים לעבוד איתו, נשאלת השאלה החשובה – איך מאמנים אותו?
 נניח יש לי רשת מהצורה

$$\begin{aligned} h_1 &= f(W_1x + b_1) \\ h_2 &= f(W_2h_1 + b_2) \\ \hat{y} &= f(W_3h_2 + b_3) \\ L &= \text{Loss}(\hat{y}) \end{aligned}$$

אז בעצם יוצא שצריך לחשב את

$$\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}, \frac{\partial L}{\partial W_3}, \frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial b_2}, \frac{\partial L}{\partial b_3}$$

אבל לדוגמה לחשב את $\frac{\partial L}{\partial b_1}$ זה מאוד בעייתי כי זה שקול לחשב את

$$\frac{\partial}{\partial b_1} \text{Loss}(f(W_3f(W_2f(W_1x + b_1) + b_2) + b_3))$$

ועבור רשתות עם יותר שכבות צריך לגזור דברים הרבה יותר מסובכים...

שאלה 1.15. איך אפשר לייעל את התהליך של חישוב הנגזרות החלקיות?

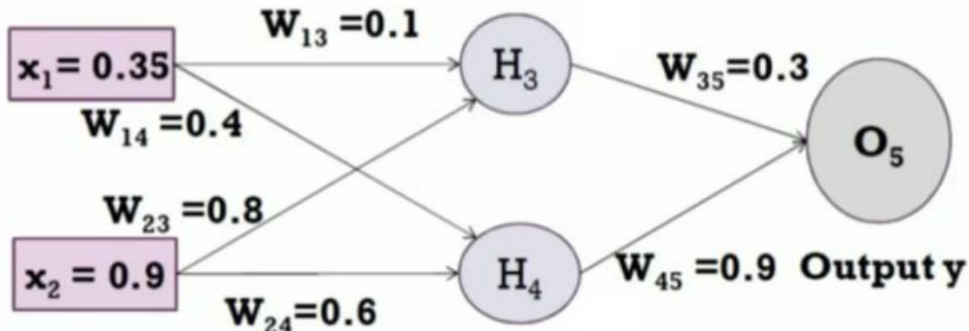
פתרון. בעזרת שימוש בכלל השרשרת!

במקום שכדי לחשב את $\frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial b_2}$ אני אחשב את הכל מהתחלה, אני יכול לכתוב

$$\begin{aligned} \frac{\partial L}{\partial b_2} &= \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_2} \frac{\partial h_2}{\partial b_2} \\ \frac{\partial L}{\partial b_1} &= \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial b_1} \end{aligned}$$

והפלא ופלא, קיבלנו בשני המקרים האלה שיש גורם משותף של $\frac{\partial L}{\partial \hat{y}}, \frac{\partial \hat{y}}{\partial h_2}$, לכן אם נשמור בצד את $\frac{\partial L}{\partial \hat{y}}, \frac{\partial \hat{y}}{\partial h_2}$, החישובים של הנגזרות יקחו פחות זמן.

דוגמה 2.15. עבור הרשת הזאת:



הניחו כי כל האקטיבציות הן $f(x) = \sigma(x) = \frac{1}{1+e^{-x}}$. בצעו מעבר *forward*, עדכנו את כל המשקולות של הרשת (מעבר *backward*), ואז בצעו עוד מעבר *forward*. הניחו כי $y = 0.5$, קצב הלמידה הוא 1, וכן $\text{Loss} = y - O_5$.

פתרון. לפי ההגדרה של נירון בודד, נקבל

$$H_3 = \sigma(x_1 w_{13} + x_2 w_{23}) = \sigma(0.35 \cdot 0.1 + 0.9 \cdot 0.8) = \sigma(0.755) = 0.68$$

באותו אופן

$$H_4 = \sigma(x_1 w_{14} + x_2 w_{24}) = \sigma(0.35 \cdot 0.4 + 0.9 \cdot 0.6) = \sigma(0.68) = 0.66$$

לכן

$$O_5 = \sigma(H_3 w_{35} + H_4 w_{45}) = \sigma(0.68 \cdot 0.3 + 0.66 \cdot 0.9) = \sigma(0.798) = 0.69$$

כלומר הפלט של הרשת הוא 0.69. לכן נקבל $Loss = 0.5 - 0.69 = -0.19$

קעת כדי לחשב את הנגזרות:

$$\frac{\partial Loss}{\partial O_5} = -1$$

$$\frac{\partial O_5}{\partial w_{i5}} = H_i \sigma(H_3 w_{35} + H_4 w_{45}) (1 - \sigma(H_3 w_{35} + H_4 w_{45})) = H_i O_5 (1 - O_5) \quad i \in \{3, 4\}$$

$$\frac{\partial O_5}{\partial H_i} = w_{i5} \sigma(H_3 w_{35} + H_4 w_{45}) (1 - \sigma(H_3 w_{35} + H_4 w_{45})) = w_{i5} O_5 (1 - O_5) \quad i \in \{3, 4\}$$

$$\frac{\partial H_i}{\partial w_{ji}} = x_j \sigma(x_1 w_{1i} + x_2 w_{2i}) (1 - \sigma(x_1 w_{1i} + x_2 w_{2i})) = x_j H_i (1 - H_i) \quad i \in \{3, 4\}, j \in \{1, 2\}$$

מכאן נקבל לפי כלל השרשרת

$$\begin{aligned} \frac{\partial Loss}{\partial w_{35}} &= \frac{\partial Loss}{\partial O_5} \frac{\partial O_5}{\partial w_{35}} = -H_3 O_5 (1 - O_5) \\ \frac{\partial Loss}{\partial w_{45}} &= \frac{\partial Loss}{\partial O_5} \frac{\partial O_5}{\partial w_{45}} = -H_4 O_5 (1 - O_5) \\ \frac{\partial Loss}{\partial w_{13}} &= \frac{\partial Loss}{\partial O_5} \frac{\partial O_5}{\partial H_3} \frac{\partial H_3}{\partial w_{13}} = -w_{35} O_5 (1 - O_5) x_1 H_3 (1 - H_3) \\ \frac{\partial Loss}{\partial w_{23}} &= \frac{\partial Loss}{\partial O_5} \frac{\partial O_5}{\partial H_3} \frac{\partial H_3}{\partial w_{23}} = -w_{35} O_5 (1 - O_5) x_2 H_3 (1 - H_3) \\ \frac{\partial Loss}{\partial w_{14}} &= \frac{\partial Loss}{\partial O_5} \frac{\partial O_5}{\partial H_4} \frac{\partial H_4}{\partial w_{14}} = -w_{45} O_5 (1 - O_5) x_1 H_4 (1 - H_4) \\ \frac{\partial Loss}{\partial w_{24}} &= \frac{\partial Loss}{\partial O_5} \frac{\partial O_5}{\partial H_4} \frac{\partial H_4}{\partial w_{24}} = -w_{45} O_5 (1 - O_5) x_2 H_4 (1 - H_4) \end{aligned}$$

כלומר

$$\begin{aligned} \frac{\partial Loss}{\partial w_{35}} &= -0.145 \\ \frac{\partial Loss}{\partial w_{45}} &= -0.141 \\ \frac{\partial Loss}{\partial w_{13}} &= -0.00488 \\ \frac{\partial Loss}{\partial w_{23}} &= -0.012 \\ \frac{\partial Loss}{\partial w_{14}} &= -0.015 \\ \frac{\partial Loss}{\partial w_{24}} &= -0.389 \end{aligned}$$

נזכור כי כלל העדכון של כל פרמטר עם GD הוא

$$w \leftarrow w - \eta \frac{\partial Loss}{\partial w}$$

ונזכור כי $\eta = 1$, לכן נעדכן את הפרמטרים שלנו לפי

$$\begin{aligned} w_{35} &= 0.3 - (-0.145) = 0.445 \\ w_{45} &= 0.9 - (-0.141) = 1.041 \\ w_{13} &= 0.1 - (-0.005) = 0.105 \\ w_{23} &= 0.8 - (-0.012) = 0.812 \\ w_{14} &= 0.4 - (-0.015) = 0.415 \\ w_{24} &= 0.6 - (-0.389) = 0.989 \end{aligned}$$

נעשה עוד מעבר $forward$:

$$H_3 = \sigma(x_1 w_{13} + x_2 w_{23}) = \sigma(0.35 \cdot 0.105 + 0.9 \cdot 0.812) = \sigma(0.768) = 0.68$$

$$H_4 = \sigma(x_1 w_{14} + x_2 w_{24}) = \sigma(0.35 \cdot 0.415 + 0.9 \cdot 0.989) = \sigma(1.035) = 0.74$$

וכן

$$O_5 = \sigma(H_3 w_{35} + H_4 w_{45}) = \sigma(0.68 \cdot 0.445 + 0.74 \cdot 1.041) = \sigma(1.073) = 0.745$$

כלומר $\eta = 1$ גרם לנו לחזות משהו יותר גרוע.

שאלה 3.15. מדוע זה קרה?

תשובה: זה בגלל שלקחנו $Loss$ להיות פונקציה שיכולה להחזיר לנו ערכים שליליים, כלומר בעדכון של ה- GD אנחנו עלונו בכיוון הגרדיאנט במקום לרדת בכיוון הגרדיאנט.

שאלה 4.15. אם היינו מחליפים את $Loss = y - O_5$ להיות $Loss = (y - O_5)^2$, האם היינו צריכים לחזור על כל החישובים שלנו מההתחלה (בחישובים הכוונה היא $forward \rightarrow backward \rightarrow forward$)?

תשובה: לא! תחילה, כל ה- $forward$ הראשון היה השאר אותו הדבר. שנית, בחישובים של הנגזרות, היינו צריכים רק לחשב מחדש את $\frac{\partial Loss}{\partial O_5}$. כמובן שזה היה מביא לנו סט חדש של פרמטרים במעבר ה- $backward$, וכאן גם ה- $forward$ השני יהיה שונה.

16 אימון רשתות (עמוקות).

בבואנו לאמן רשת נוירונים, ישנם מספר בעיות שחוזרות על עצמן שוב ושוב, ולכן חשוב להכירן.

16.1 Vanishing\Exploding gradients

נניח ועכשיו אני מאמן רשת נוירונים עם 50 שכבות, ואני רוצה לעשות צעד $back-propagation$. אם הרשת שלי נראית משהו כמו

$$h_{n+1} = f(W_{n+1}h_n + b_{n+1})$$

אז אני אצטרך להכפיל בנגזרת שלי הרבה מאוד $\frac{\partial h_i}{\partial h_{i-1}}$, והבעיה שלי היא שברגע שאני מכפיל הרבה מאוד מספרים, המכפלה יכולה לעשות 2 דברים:

1. לקרוס ל-0: אם המוכפלים קטנים מ-1.

2. להתפוצץ ל- ∞ : אם המוכפלים גדולים מ-1.

כמובן ששתי התוצאות האלה לא טובות לנו בזמן האימון, כי אם תהיה לי נגזרת מאוד מאוד קטנה אז כשאני אעדכן את המשקולות שלי אני אראה

$$W' = W - \underbrace{\eta \frac{\partial Loss}{\partial W}}_{\approx 0} = W$$

כלומר לא עדכנתי את W , גם אם אני עדיין רחוק מלהגיע למינימום של ה- $Loss$. אם תהיה לי נגזרת מאוד גדולה אזי אני אעדכן את W להיות משהו מאוד גדול ואז כל הרשת שלי תתפוצץ ותפלוט סתם ערכים גדולים.

הפתרון יהיה לאתחל את המשקולות שלי בצורה חכמה!

16.1.1 Weight Initialization

הרעיון הוא שאנחנו רוצים שהשונויות של הקלט יהיה שווה לשונויות של הפלט כדי שהשכבה לא תתפוצץ/תדעך. אם יש לי $y = Wx$ אזי

$$\mathbb{V}[y_i] = \mathbb{V}\left[\sum_{j=1}^{d_{in}} x_j w_j\right] \stackrel{*}{=} d_{in} \mathbb{V}[x_i w_i] \stackrel{**}{=} d_{in} \left(\mathbb{E}[x_i^2] \mathbb{E}[w_i^2] - \mathbb{E}[x_i]^2 \mathbb{E}[w_i]^2 \right) \stackrel{***}{=} d_{in} \mathbb{V}[x_i] \mathbb{V}[w_i]$$

לכן אם ניקח $\mathbb{V}[w_i] = \frac{1}{d_{in}}$ נקבל $\mathbb{V}[y_i] = \mathbb{V}[x_i]$.

* נניח x, w בלתי תלויים שויי התפלגות.

** נניח $x \perp w$.

*** נניח x, w בעלי תוחלת 0.

נעיר כי זו לא שיטה חסרת תקלות: הרי אנחנו מניחים ש- w ממורכז סביב 0, אבל אחרי שכבת $ReLU$ אחת, כל מה שמתחת ל-0 יהפוך ל-0 ומה שאומר שלא יהיו תוצאות שליליות....

המחקר בתחום הזה עדיין מאוד פעיל וכל שנה צצות שיטות חדשות איך לאתחל את המשקולות בצורה נכונה.

עוד דרך בה אפשר להגיע לבעיה הזו היא בעזרת פונקציות אקטיבציה שנהיות רוויה: קחו לדוגמה את $\sigma(x) = \frac{1}{1+e^{-x}}$. מה קורה עבור $x = \pm 4$? כבר אז אנחנו מקבלים תוצאה שקרובה ל-1, לכן הנגזרת, שהיא $\sigma(x)(1 - \sigma(x))$ תהיה קרובה ל-0.

הפתרון: להשתמש בפונקציית אקטיבציה שלא נהיית רוויה, כמו $ReLU$ ושאר הפונקציות מאותה משפחה.

דרך שלישית לגרום לנגזרות לא להתפוצץ היא בעזרת *Batch Normalization*: ניקח *batch* מהדגימות שלנו בשלב מסוים ונחשב להם

$$\mu_B = \frac{1}{m_B} \sum_{i=1}^{m_B} x^i$$

$$\sigma_B^2 = \frac{1}{m_B} \sum_{i=1}^{m_B} (x^i - \mu_B)^2$$

ואז נעשה

$$\hat{x}^i = \frac{x^i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

על מנת לתת לרשת ללמוד את ההתפלגות האופטימלית של הדאטה, נציג 2 פרמטרים חדשים לתהליך הלמידה: β, γ ואז נחשב

$$z^i = \gamma \hat{x}^i + \beta$$

וניתן לרשת ללמוד גם אותם.

את כל התהליך הזה עושים מיד לפני/ מיד אחרי פונקציית האקטיבציה של השכבה החבויה, ועושים אותו בכל השכבות.

16.1.3 Gradient Clipping

הפתרון הרביעי שאנחנו יכולים לנסות הוא *Gradient Clipping*: להכריח את הנגזרת שלנו להיות עד טווח מסוים. נניח אני לא רוצה נגזרת שבערך מוחלט תהיה מעל 5, אז אני אקבע שכל נגזרת שלי תהיה

$$dw = \begin{cases} -5 & dw < -5 \\ dw & -5 \leq dw \leq 5 \\ 5 & 5 < dw \end{cases}$$

וכך אני יכול באופן מלאכותי לשלוט על גודל הנגזרות שלי. ככה למשל, הגרדיאנט

$$(-6, 0.7, 5.2, 3.4, 8)$$

יהפוך ל-

$$(-5, 0.7, 5, 3.4, 5)$$

אבל נשים לב כי איבדנו את הכיוון של הגרדיאנט הזה, לכן אם נרצה לשמר את הכיוון הזה, לא נקבע מקסימום לערכים, אלא מקסימום לנורמה, וננרמל את הגרדיאנט כדי שנגיע למקסימום הזה $\nabla w = \nabla w \frac{thr}{\|\nabla w\|}$.

16.2 Dropout - רגולריזציה ברשתות נוירונים

עכשיו נניח שהרשת מתאמנת, אבל יש *Overfit*! איך אנחנו יכולים למנוע מהרשת לשנן את הדאטה, וללמוד אותו במקום? אפשר לעשות *dropout*: הרעיון הוא שבכל *forward*, נכבה ברנדומליות נוירונים ברשת (נקבע אותם להיות 0). ההסתבכות לכיבוי היא היפרפרמטר - לרוב נהוג 0.5. הרעיון כאן היה ששום נוירון לא צריך "להשתלט" על הרשת ולצבור יותר מדי כוח לבדו, ושם יש נוירון כזה והוא יכבה, הרשת תצטרך ששאר הנוירונים ילמדו במקומו וככה לא יסתמכו עליו. חשוב שאותו פרמטר כיבוי יבכר לאורך כל השכבות, ולא שיהיה פרמטר אחר לכל שכבה.

17 Optimizers

אנחנו יודעים שכדי לאמן מודל משתמשים ב-*GD*, אבל על מנת לאמן רשת נוירונים צריך כמות גדולה של דאטה, מה שאומר שלפעמים לעשות *GD* יקח הרבה מאוד זמן. לכן אנחנו מחפשים שיטות אחרות לאמן את המודל שלנו, כל שיטה כזו נקראת *optimizer*.

דוגמה 1.17. *SGD* הוא *optimizer* אחר, שעובד יותר מהר מ-*GD* במחיר שפה אנחנו לא עושים גרדיאנט על כל הדאטה, אלא על חלק ממנו. כדי לפתור חלק מהבעיות של *SGD* הציעו להכניס מומנטום לעדכון:

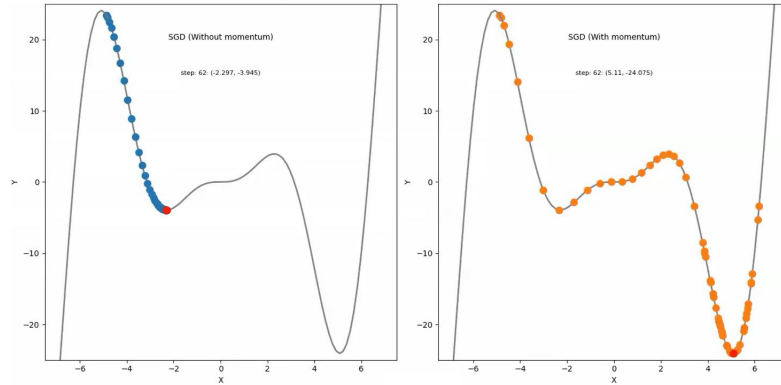
17.1 Momentum

הרעיון הוא שאם יש לנו ציר אחד שהוא מאוד איטי וציר אחד שהוא מאוד מהיר, אז היינו רוצים שבציר האיטי כל עוד אני נע באותו כיוון שאני אצבור שמה מהירות ואנוע יותר מהר. לעומת זאת אם אני כל איטרציה נע בכיוון אחר אז אני לא צריך לקבל מומנטום.

$$x_{t+1} = x_t - \alpha v_{t+1}$$

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

הרעיון פה זה שכל שאני יותר איטרציות עם $\nabla f(x_t)$ בכיוון של v_t , ככה v_{t+1} ימשיך לעלות ואז ההתקדמויות יהיו מהירות יותר ויותר באותו ציר. מומנטום אפילו יכול לעזור לברוח ממינימום מקומי!



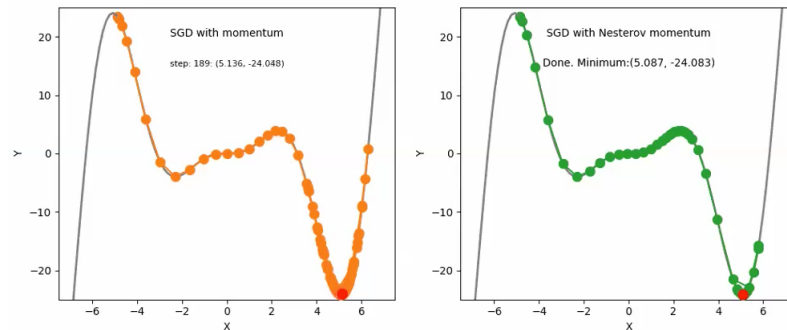
בעיות: יכול להיות ש- v יצבור יותר מדי מהירות ואז יעבור את המינימום המקומי ובאיטרציות הבאות נצטרך לחזור לכיוון הנגדי כדי להגיע למינימום.

17.2 Nesterov Momentum

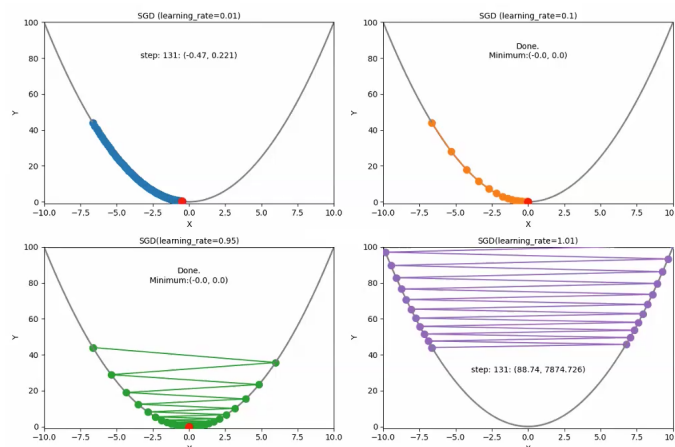
במקום לחשב את הגרדיאנט ב- x_t שלי, אני מחשב את הגרדיאנט אפה שאני אמור להיות ($x_t + \rho v_t$), וככה אני "מסתכל קדימה" על הנקודה שמומנטום יקח אותי אליה כדי להבין אם אני הולך לפספס מינימום לוקאלי. בנוסחאות זה

$$x_{t+1} = x_t + v_{t+1}$$

$$v_{t+1} = \rho v_t - \alpha \nabla f(x_t + \rho v_t)$$



17.3 AdaGrad



בתמונה רואים השוואה בין 4 קצבי למידה: 2 גדולים ו-2 קטנים. אם ה- lr גבוה אז העדכוניים שלנו יקפצו ממקום למקום. אם נקבע את ה- lr להיות 0.1 אז נגיע למינימום לאט, ואם נקבע אותו להיות 0.01 נגיע אליו ממש ממש לאט. איך אני יודע אם קבעתי lr גבוה מדי/ נמוך מדי? באותו אופן בדיוק, אם המודל שלי מתאמן ואני רואה את ה- $loss$ מקפץ אז ה- lr גבוה מדי ואם ה- $loss$ לא זז אז ה- lr נמוך מדי. עכשיו מה קורה אם יש לי ציר שבו ה- lr גבוה מדי ובציר אחר ה- lr נמוך מדי? בשביל זה המציאו את *AdaGrad* שבא להתאים lr שונה לכל ציר. בנוסחאות זה

$$v_t = v_{t-1} + (\nabla f(w_t))^2$$

$$x_{t+1} = x_t - \frac{\alpha}{\sqrt{v_t + \varepsilon}} \nabla f(w_t)$$

החילוק של α ב- $\sqrt{v_t + \varepsilon}$ אומר שבכיוונים שאני ממש מהיר בהם ה- lr הכולל יקטן (כי v_t יהיה גדול) ובכיוונים שאני איטי בהם ה- lr הכולל יגדל (כי v_t יהיה קטן).

הבעיה: v_t תמיד עולה ועולה, כלומר ה- lr תמיד קטן, ויש מצב שניעצר ונגיע ל- $lr = 0$ ועוד לא הגענו למינימום.

17.4 RMSProp

נועד למנוע את הבעיה של *AdaGrad* שממשיכים לחלק את α בערך שרק עולה. עושה את זה בדרך הבאה:

$$x_{t+1} = x_t - \frac{\alpha}{\sqrt{v_t + \varepsilon}} \nabla f(w_t)$$

$$v_t = \beta v_{t-1} + (1 - \beta)(\nabla f(w_t))^2$$

β נקרא *decay rate*.

הרעיון פה הוא שכל עוד $(\nabla f(w_t))^2$ גדול, v_t ילך וימשיך לגדול, אבל אם אנחנו לא זזים ממש מהר אז הגורם הדומיננטי הוא βv_{t-1} כלומר v_t יקטן בחזרה.

17.5 Adam

זה שילוב של *RMSProp* ו-*Momentum*, גם להאיץ את הלמידה בצירים שצריך אבל גם לבטל את הרעש שיש בזה ולא לעשות יותר מדי *overshot*. הנוסחאות יותר מסובכות:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla f(w_t) \quad \text{Momentum}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla f(w_t))^2 \quad \text{RMSProp}$$

היינו רוצים לפשט את הנוסחא של *Adam* ולכתוב

$$x_{t+1} = x_t - \frac{\alpha}{\sqrt{v_t + \varepsilon}} m_t$$

אבל יש עניין טכני קטן שאומר ש- m_t, v_t נוטים להיות מוטים לעבר 0 בצעדים הראשונים, לכן צריך לתקן אותם לפי

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

(β_i^t אומר β_i בחזקת t) ואז אפשר לכתוב

$$x_{t+1} = x_t - \frac{\alpha}{\sqrt{\hat{v}_t + \varepsilon}} \hat{m}_t$$